RAKSHA - Real-time AI Knowledge Screening & Hazard Audit

Version 1.5

YP-Raksha

TECHNICAL MANUAL

YP-Raksha Technical Documentation

Architecture, APIs, data model, security controls and Browser AI implementation

Contents and document index

Section page numbers include the cover. Headings are listed in reading order.

SECTION	PAGE
Document purpose audience and scope	**5**
Version history	5
System overview	**5**
Problem statement and intended outcomes	6
Product use cases and implementation outcomes	6
Technology stack	**7**
Repository layout	**7**
Core module	**8**
Providers	8
Schemas	8
MCP client	8
Key selection and networking	8
Framework module	**9**
ConfigStore	9
LogsStore	9
RBAC	9
Other framework packages	10
Plugins	**10**
HTTP surface	**11**
Entry point and flags	11
Middleware assembly	11
Request identification on inference	12
Route groups	12
Health and observability endpoints	13
Data model key tables	**13**
ConfigStore tables	13
Browser AI tables	14
Browser AI Guard technical detail	**14**
Backend routes	14
Guard fleet authentication no user session	15
Guard desktop agent	16
mitmproxy addon	16
Guard distribution and hot update	17
Frontend	**17**
Build and integration	17
Pages	17
Sidebar and section gating	18
Browser AI UI implementation files	18
Security model	**18**
Build test and developer workflow	**19**
Gateway	19
UI	19
Guard	19
Docker Helm	20
Conventions	20
Extension points	**20**
Glossary	**21**
Enterprise deployment context	**21**
Client deployment profile at a glance	22

SECTION	PAGE
Implementation boundaries relevant to product selection	**22**
Operational security review points	**23**
Product architecture and request flows	**23**
End to end product architecture	24
Inference request lifecycle	25
Browser AI Guard decision lifecycle	26
Administration UI and policy change lifecycle	27
Optional MCP tool lifecycle	28
Observability and settings to policy propagation	29
State scaling and component ownership	29
Error handling retries and recovery	**30**
Error response contract	30
Common failure classes and caller action	30
Retry key rotation and fallback behavior	31
Streaming WebSocket and partial response errors	32
Operational recovery matrix	32
Incident evidence and escalation	33
Error handling decision flow	33
Deployment prerequisites and configuration	**33**
Required platform dependencies	33
Runtime configuration groups	34
Production packaging cautions	35
Docker and container deployment procedure	35
Database provisioning and lifecycle	36
Browser Guard agent and extension installation	37
Release promotion and rollback	**38**
Feature and functional reference	**38**
User roles and permissions	39
Representative workflows	39
Logical data design and lifecycle	**40**
API documentation and integrations	**41**
Illustrative inference call	42
Illustrative Guard event	42
Errors quotas and integrations	42
Security and compliance mapping	**43**
Deployment environments and infrastructure	**43**
Environment model and release pipeline	43
Runtime prerequisites and compatibility boundaries	44
Performance and scalability	**45**
Monitoring backup and maintenance	**45**
Testing QA and known limitations	**46**
Troubleshooting and support	**46**
Appendix configuration diagrams and roadmap	**47**
Environment configuration inventory example	47
Diagram index	47
Roadmap and future enhancements	47
End user quick manual	**48**
Employee using a managed browser	48
Employee using the YP Raksha workspace	48
Application developer using the Gateway API	48
Quick troubleshooting	49
Administrator operating manual	**50**
Establish an environment	50

SECTION	PAGE
Secure administrator access and roles	50
Configure provider model and application access	50
Configure Browser AI Guard	50
Configure observability retention and service ownership	51
Change backup restore and recovery operations	51
Manage workspaces and optional integrations	51

Document purpose audience and scope

This manual explains YP-Raksha architecture, request and event flows, stored data, security controls and on-premises deployment. It is for enterprise architecture, application, database, platform, network security, endpoint, identity, operations and assurance teams planning a controlled implementation.

YP-Raksha combines an AI Gateway for application APIs with Browser AI Guard for managed browser use. It evaluates configured policies, governs model routing and records operational and audit events. Behavior depends on the selected release, enabled components and bank configuration; qualify it with the release package and UAT evidence.

Scope: Gateway and embedded UI, Guard agent and extensions, APIs and integrations, data, security, deployment and operations. This implementation reference does not replace a bank-approved design, a release support statement or UAT/load/DR evidence. Commercial terms and certification claims are outside its scope.

Glossary terms and abbreviations used in the codebase are listed in Section 14.

Version history

Document revision	Date	Summary
1.5	24 October 2026	Client-facing technical reference for YP-Raksha Gateway and Browser AI Guard.

The cover and table identify document issue 1.5. Record the exact product release, signed artifact digests and compatibility matrix in the bank's controlled deployment records.

System overview

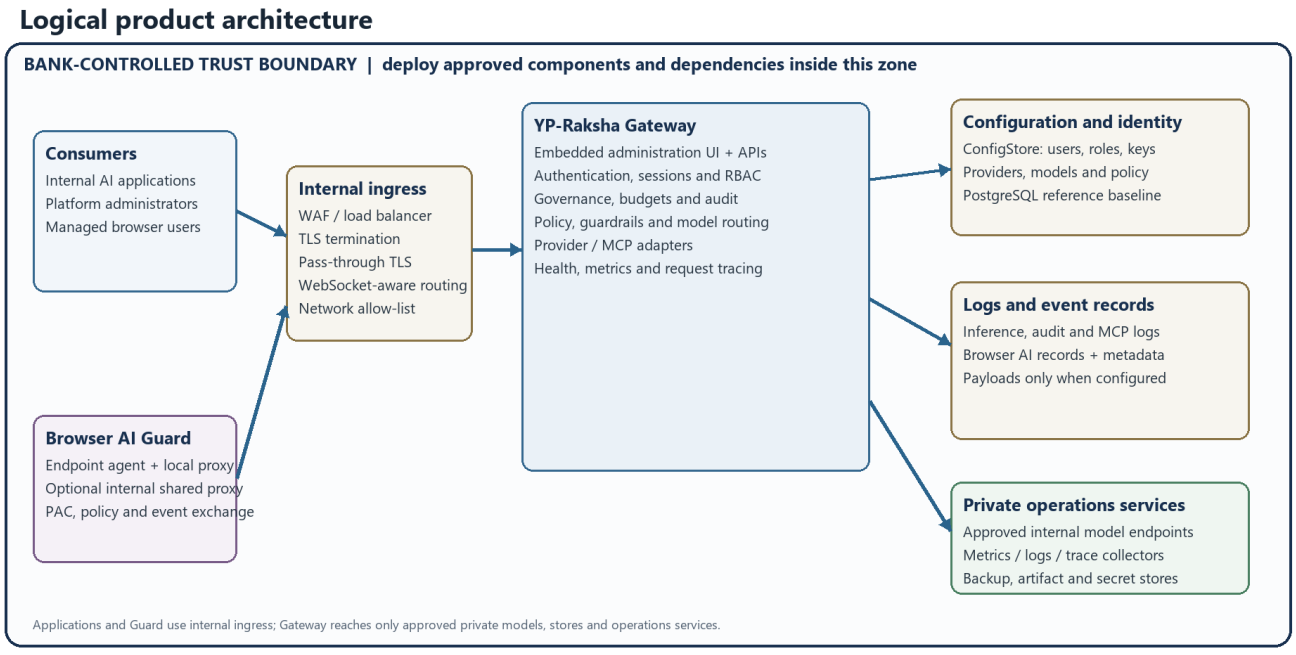


Figure 1. Logical YP-Raksha architecture. Optional services and destinations depend on configuration; the bank must enforce the residency boundary.

At a glance, `/v1/*` is the inference data plane for governed request/response handling, while `/api/*` is the configuration and management control plane for the dashboard, RBAC, observability and Browser AI Guard.

Problem statement and intended outcomes

Organizations can have AI requests flowing through application-specific integrations and employee browsers without a common point for approved model routing, policy enforcement, access control and review. This can create inconsistent controls, unclear data destinations, unmanaged provider credentials and limited incident evidence. YP-Raksha provides a customer-operated Gateway and managed-browser control path so the organization can apply centrally configured controls and retain configured records in its own environment.

YP-Raksha does not independently make an environment private, guarantee that every browser surface is inspected, or replace network egress controls, identity governance, endpoint management or the bank's data-retention policy. Those outcomes require the deployment boundaries and release-specific qualification described in this manual and the On-Premises Deployment Architecture.

Product use cases and implementation outcomes

Use case	Primary actor	YP-Raksha path	Expected outcome and boundary
Govern internal application inference	Application owner and platform administrator	Application → internal ingress → Gateway authentication/governance → configured provider adapter → approved model	Central route, key, budget/rate and configured policy controls; exact operation support depends on the provider adapter.
Reduce sensitive prompt submission from managed browsers	Employee, endpoint team and security administrator	Managed browser → Guard extension/agent or approved shared proxy → target/rule evaluation → configured action → internal event API	Supported captured prompt/upload surfaces can be allowed, redacted, warned or blocked according to configuration; network policy remains necessary for mandatory deny.
Investigate AI policy events	Security operations	Authorized Browser AI pages and logs → scoped management API → internal database/collector	Review permitted records and event context under access and retention policy; payload capture is configuration-dependent.
Control model spend and access	Application owner and governance administrator	Virtual key/user/team scope → budget/rate controls → provider/model route	Apply configured caller limits and route permissions; provider pricing and accounting depend on maintained model/pricing configuration.
Operate a bank-resident deployment	Platform, network, DBA and security teams	Private model, internal persistence/identity/observability/ artifacts, bank backup and DR	Can support a bank/India-only design when all external routes are denied and all dependencies and copies stay inside the approved boundary.
Govern approved MCP tool access	Application owner and MCP administrator	Client/session → Gateway tool authorization → configured private MCP server → result/log path	Controls apply to configured tools and destinations; each server remains an explicitly approved data destination.

The deployment reference includes the 120,000-endpoint workload assumptions and site resource envelope. This is a qualification design, not a measured product score, benchmark, certified maximum or SLA. The product fit scorecard in the Product Datasheet distinguishes implementation evidence from release qualification and contractual support.

Technology stack

Layer	Technology	Source of truth
Gateway	Go 1.26.4, `valyala/fasthttp`, `fasthttp/router`, `automaxprocs`	`go.work`, `transports/gateway-http/main.go`
Persistence	GORM; Postgres / MySQL / SQLite (CGO `go-sqlite3`, `sqlite_static`)	`framework/configstore`, `framework/logstore`
Provider SDKs	Vendor HTTP clients + official SDKs	`core/providers/*`
Plugins	Go interfaces, optional `.so` dynamic loading (`SharedObjectPluginLoader`)	`plugins/`, `server/plugins.go`
MCP	Model Context Protocol client/server, OAuth2 sessions	`core/mcp`, `handlers/mcp*.go`
Frontend	React 19, TypeScript 5.9, Vite 8, TanStack Router/Table, Redux Toolkit, Radix UI, Tailwind 4, Monaco, Recharts, Zod	`ui/package.json`
Guard agent	Python 3.11+, mitmproxy addon, PyInstaller, Inno Setup (Windows), pkg/dmg scripting (macOS)	`apps/browser-guard/**`
Deploy	Docker (multi-stage), docker compose, Helm (Postgres, Redis, Qdrant, Weaviate, HPA, Ingress)	`deploy/**`, `docker-compose.yml`
Tooling	Make targets, gotestsum, Playwright, newman, oxlint/oxfmt	`Makefile`

Repository layout

```
core/                       Go module: the AI engine
  providers/<vendor>/       one package per provider (chat, responses, images, audio, video, batch…)
  schemas/                  shared types (requests, responses, plugin interfaces, errors)
  mcp/                      MCP client, code mode, credential store
  keyselectors/             weighted/random key selection strategies
  network/                  HTTP client utilities (retries, timeouts, proxies)

framework/                  Shared infrastructure
  configstore/              ConfigStore + WorkspaceStore; tables/ has every GORM model
  logstore/                 LLM/MCP/Browser AI log storage, aggregations, matviews, cleaners
  rbac/                     Resource×Operation enforcement, route → requirement mapping
  encrypt/                  AES encryption at rest for secrets in DB
  cluster/, loadbalancer/, routing/, circuitbreaker/, kvstore/, featureflags/
  connectors/, objectstore/, vectorstore/, oauth2/, mailer/, alerts/, tracing/
  migrator/, mysqlconn/, postgresconn/    DB drivers + schema migration helpers

plugins/                    telemetry, prompts, logging, governance, otel, semanticcache,
                            compat, maxim, guardrails, connectors, modelcatalogresolver,
                            mocker, jsonparser

transports/gateway-http/     The server binary
  main.go                   flags, logger, version banner, bootstrap/start
  server/server.go          GatewayHTTPServer: bootstrap, middleware assembly, graceful shutdown
  server/plugins.go         built-in plugin instantiation + ordering
  handlers/*.go             every /api and /v1 handler (one file per feature area)
  integrations/*.go         pass-through provider surfaces
  websocket/, ui/           WS pool/sessions; embedded UI assets

ui/                         React dashboard (Vite). pages under ui/app/workspace/**
apps/browser-guard/         Python Guard agent + mitmproxy addon + installers + release artifacts
configs/                    config.json, mcp-library.json, model-parameters.json, pricing.json
deploy/                     docker/, helm/gateway/
docs/                       this documentation set
```

Core module

Providers

Each directory under `core/providers/<vendor>` implements the shared provider contract from `core/schemas`. A provider package contains files such as `chat.go`, `responses.go`, `images.go`, `speech.go`, `transcription.go`, `videos.go`, `batch.go`, `models.go`, `errors.go`, `types.go`, `utils.go`.

Capability differences that matter (from the files present):

Provider(s)	Notable capability files
`openai`	chat, responses, responseslifecycle, text, images, speech, transcription, videos, batch, files, embedding, realtime, websocket, large_payload, passthrough_usage
`anthropic`, `gemini`	chat, responses, count_tokens / models, embedding (gemini), types
`vertex`	chat, batch, cachedcontents, count_tokens, embedding, rerank, models
`bedrock`, `bedrockmantle`	chat, responses, embedding, images, batch
`replicate`, `runware`, `runway`	chat, images, videos, files
`mistral`	adds `ocr.go` on top of chat/transcription/cachedcontents
`elevenlabs`	audio-only
`ollama`, `sgl`, `vllm`, `openrouter`, `parasail`, `opencode`, `groq`, `cerebras`, `deepseek`, `fireworks`, `huggingface`, `nebius`, `perplexity`, `xai`, `cohere`	vendor subsets (mostly OpenAI-compatible shapes)
`openaicompat`	generic OpenAI-compatible provider + `registry.go` for many compatible endpoints
`azure`	Azure OpenAI, deployment-aware

Shared helpers live in `core/providers/utils`: `decompression.go`, `sse.go`, `stream.go`, `streamterminaldetector.go`, `pagination.go`, `images.go`, `audio.go`, `file.go`, `videos.go`, `bodysigner.go` (request body signing for AWS-style auth), `modelparamscache.go`, `largeresponse.go`, `passthrough_stream.go`, `fetch.go`, `models.go`, `utils.go`.

Schemas

`core/schemas` holds the types shared by the gateway and every plugin: chat / responses / text / embedding / image / audio / video request-response types, `GatewayConfig`, `GatewayContext`, `ProviderConfig`, `NetworkConfig`, the plugin interfaces (`BasePlugin`, `LLMPlugin`, `MCPPlugin`, `ObservabilityPlugin`, `HTTPTransportPlugin`, `GatewayHTTPMiddleware`), error types and log level/output enums.

MCP client

`core/mcp` implements the Model Context Protocol client used by the MCP Gateway: connection lifecycle, tool discovery, tool execution, per-user credentials (`credstore`), "code mode" (`codemode`) and utilities. The HTTP layer wraps it in `handlers/mcp.go` (clients, library, tool groups), `handlers/mcpserver.go`, `handlers/mcpsessions.go` and the OAuth2 set (`mcpoauth2.go`, `mcpoauth2discovery.go`, `mcpoauth2issuance.go`, `mcpoauth2consent.go`, `mcpoauth2sessions.go`, `mcpoauth2jwt.go`).

Key selection and networking

- `core/keyselectors` — weighted/random selection across multiple keys per provider.
- `core/network` — the outbound HTTP client for provider calls (proxy, TLS, timeouts, retries),

driven by each provider's `NetworkConfig`.

Framework module

ConfigStore

`framework/configstore` is the configuration database layer.

- Drivers: `postgres.go`, `mysql.go`, `sqlite.go`, `rdb.go`; interface in `store.go`; errors in `errors.go`.
- `migrations.go` plus `framework/migrator` (`addcolumn.go`, `dropcolumn.go`) handle schema evolution.
- `tables/` holds one file per domain: `user.go`, `workspace.go`, `enterprise.go`, `team.go`,

`customer.go`, `virtualkey.go`, `key.go`, `provider.go`, `model.go`, `modelconfig.go`, `modelpricing.go`, `modelparameters.go`, `pricingoverride.go`, `budget.go`, `ratelimit.go`, `routingrules.go`, `mcp*.go`, `mcpoauth2*.go`, `prompts.go`, `promptVersions.go`, `promptSessions.go`, `skills.go`, `folders.go`, `featureflag.go`, `plugin.go`, `config.go`, `clientconfig.go`, `env.go`, `sessions.go`, `temptokens.go`, `dlock.go` (distributed locks), `logstore.go`, `vectorstore.go`, `smtp.go`, `confighash.go`.

- `workspace.go` seeds the **RBAC catalog**, the three **system roles**, and exposes the

resource/operation/name catalogs used by backend and UI.

- Secrets in the DB are encrypted with `framework/encrypt` when `GATEWAY_ENCRYPTION_KEY` is set

(`tables/encryption.go`, `configstore/encryption.go`).

LogsStore

`framework/logstore` stores and aggregates request logs.

- Files: `logger.go`, `store.go`, `rdb.go`, `postgres.go`, `mysql.go`, `sqlite.go`, `hybrid.go`.
- `browser_ai.go` is the **entire Browser AI persistence layer** (see §9).
- Aggregations for dashboard/log pages: `matviews.go` (materialized views), `tables.go`,

`cleaner.go` (retention), `asyncjob.go` (async inference jobs), `payload.go`.

RBAC

1. **Resource × Operation** — `framework/rbac/enforce.go`, `PathRequirementFor(method, path)`

returns the requirement for a route (or `nil` = not RBAC-gated). `admin` resolves to *allow all*; an unknown/unseeded role yields an **empty set = deny**. `View` and `Read` are interchangeable.

2. **Sidebar section grants** — `framework/rbac/sections.go`, `SectionRequirementFor(method, path)`,

used for section-scoped sessions. Keys are `parent` or `parent/child`; `parent/*` accepts the parent or any child. Legacy `observability/browser-ai` is normalised to `browser-ai`.

Resource catalog (33): `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning,

Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins, MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance, RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys, Inference, Metrics, FeatureFlags, CircuitBreaker`

Operation catalog (6): `Read, View, Create, Update, Delete, Download`

System roles (auto-seeded in `configstore/workspace.go`, then merged on every boot so new permissions are picked up — except for roles a customer has customised):

Role	DAC	Grants
admin	all-data	every permission
sub_admin	all-data	Read/View on Dashboard, Inference, Observability, MCPLogs, RoutingRules, Metrics; **full** (Read/View/Create/Update/Delete/Download) on Logs — this is what gates all Browser AI write APIs; full on VirtualKeys, PromptRepository; View/Read/Update on Governance; View/Read/Create/Update on MCPGateway, MCPToolGroups; View/Read on ModelProvider. **No** master provider keys, **no** system settings
user	own-data	Read/View **only**, on Dashboard, Logs, Inference, PromptRepository, Observability, MCPGateway (the readIDs set). The UI labels this "Prompt Repository only"; the backend also allows read-only views of logs/observability/MCP. Use allowed_sections to narrow what a user actually sees

Custom roles are created in the UI. admin is always re-synced to the full catalog.

Browser AI note: every /api/browser-ai/* route maps to the **`Logs`** RBAC resource (PathRequirementFor), so a role needs Logs: View/Update (or the browser-ai section grant) to see the Browser AI pages.

Other framework packages

Package	Purpose	API surface
cluster	multi-node config propagation	/api/cluster, /internal/cluster/kv
loadbalancer	adaptive routing across providers/keys	/api/load-balancer, /api/load-balancer/routes
routing	routing-rule engine	/api/governance/routing-rules, /api/routing-rules
circuitbreaker	failure isolation policies	/api/circuit-breaker/policies, /state
featureflags	server-side flags	/api/feature-flags
rbac, oauth2, temptoken	permissions, OAuth2 provider, short-lived tokens	/api/rbac/*, /api/scim/oauth/*
mailer, alerts	SMTP and alert delivery (webhooks)	/api/smtp-config, /api/alert-channels
connectors	external sinks (BigQuery, Kafka, Datadog, New Relic, PubSub)	/api/connectors
objectstore, vectorstore, kvstore	blob / vector / KV backends (Qdrant, Weaviate, Redis)	/api/vector-store-config, /api/cache
tracing	trace store + spans	/api/logs trace views
queryscope, mcpcatalog, mcptoolgroups, mcp_headers, modelcatalog, encrypt, envutils	supporting services	–
streaming	SSE/stream utilities shared by transports	–

Plugins

A plugin implements one or more core interfaces and is loaded either as a **built-in** (by name) or as a **dynamic `.so`** via SharedObjectPluginLoader (server/plugins.go).

Built-in load order (fixed placement indexes):

#	Plugin	Role
1	telemetry	request metrics/counters
2	prompts	prompt repository integration
3	logging	LLM/MCP log persistence
4	governance	virtual keys, budgets, rate limits, teams, customers, business units
5	otel	OpenTelemetry export
6	semanticcache	embedding-based response caching
7	compat	parameter compatibility/conversion between client and provider shapes
8	maxim	Maxim observability integration
—	guardrails	rule-based input/output validation + provider config
—	connectors	external log/analytics sinks
—	modelcatalogresolver	resolves model metadata/pricing
—	mocker	mock provider for tests/E2E
—	jsonparser	tolerant JSON parsing helpers

Plugin config lives in `config.json` under `plugins`; plugins can be enabled/disabled and reordered at runtime (`GET`/`POST /api/plugins`). A plugin that fails to load is marked disabled and reported in the startup plugin-status table (`plugin status: <name> - <status>`).

HTTP surface

Entry point and flags

`main.go`:

Flag	Env fallback	Default
-port	APP_PORT	Protected deployment setting; not published here
-host	APP_HOST	Protected deployment setting; not published here
-app-dir	APP_DIR	./data
-log-level	LOG_LEVEL	info
-log-style	LOG_STYLE	json

Startup prints an ASCII banner (`-ldflags -X main.Version=…`), starts the pprof server, configures the logger, then `Bootstrap()` → `Start()`.

Middleware assembly

Server handler wrapper (`server.go`):

```
SecurityHeadersMiddleware → CORS → RequestDecompression → router
```

API middleware chain (`apiMiddlewares`, applied to every `/api/*` route):

```
AuthMiddleware.APIMiddleware()      session cookie / Bearer session token
WorkspaceAuditMiddleware            audits mutating /api/* into Audit Logs
RBACMiddleware                      resource × operation + section grants
```

Inference middleware chain (`/v1/*` and pass-through):

```
TracingMiddleware → TransportInterceptor → AuthMiddleware.InferenceMiddleware → handler
```

`TransportInterceptor` runs `HTTPTransportPostHook` plugins and captures their log entries; the tracing defer runs afterwards so plugin logs land inside the trace.

Request identification on inference

- `Authorization: Bearer <virtual-key|provider-key>` or `x-api-key: <key>`.
- Dashboard-originated inference is enriched from the session cookie

(`enrichInferenceFromDashboardSession`); provider headers are normalised to a single `Authorization: Bearer …` (`x-api-key` is deleted).

- Password-as-Bearer (base64 `user:pass`) is **removed** — only session tokens are accepted as Bearer.
- Virtual Key self-service quota: `GET /api/governance/virtual-keys/quota` (the VK header is the credential).

Route groups

Group	Representative routes	Handler file(s)
Health / UI	`GET /health`, `GET /robots.txt`, `/` (SPA)	`health.go`, `ui.go`
Session / auth	`/api/session/login`, `/logout`, `/register`, `/register/verify`, `/register/resend`, `/forgot-password`, `/verify-otp`, `/reset-password`, `/forgot-username`, `/is-auth-enabled`, `/ws-ticket`, `/api/session/users*`	`session.go`, `session_register_verify.go`, `auth_security.go`
Workspace / RBAC	`/api/users/{id}/role`, `/api/roles*`, `/api/permissions`, `/api/resources`, `/api/operations`, `/api/rbac/*`, `/api/access-profiles*`	`workspace.go`, `rbac.go`, `rbac_enforce.go`, `accessprofiles.go`
Governance	`/api/governance/virtual-keys*`, `/customers*`, `/teams*`, `/business-units*`, `/routing-rules*`, `/complexity-analyzer-config`, `/api/guardrails/*`	`governance.go`, `businessunits.go`, `team_members.go`, `guardrails.go`
Inference	`/v1/chat/completions`, `/v1/completions`, `/v1/responses*`, `/v1/embeddings`, `/v1/rerank`, `/v1/ocr`, `/v1/audio/*`, `/v1/images/*`, `/v1/videos*`, `/v1/batches*`, `/v1/files*`, `/v1/containers*`, `/v1/realtime`, `/v1/async/*`	`inference.go`, `asyncinference.go`, `realtime_*.go`, `webrtc_realtime.go`
Pass-through	`/openai/*`, `/anthropic/*`, `/genai/*`, `/bedrock/*`, `/cohere/*`, `/cursor/*`, `/litellm/*`, `/langchain/*`, `/pydanticai/*`	`integrations/*.go`
Observability	`/api/logs*`, `/api/logs/dashboard`, `/api/logs/filterdata`, `/api/logs/rankings*`, `/api/logs/histogram/*`, `/api/mcp-logs*`, `/api/audit-logs`, `/api/connectors`, `/api/alert-channels`	`logging.go`, `auditlogs.go`, `alertchannels.go`, `integrations.go`
Providers / models	`/api/providers*`, `/api/keys*`, `/api/models*`, `/api/model-configs*`	`providers.go`, `provider_keys.go`, `list_models_vk.go`
MCP	`/api/mcp/clients`, `/api/mcp/library*`, `/api/mcp/client*`, `/api/mcp/tool-groups*`, `/api/mcp-logs`, `/api/mcp/inference`	`mcp.go`, `mcpserver.go`, `mcpsessions.go`, `mcptoolgroups.go`, `mcpheaders.go`
Prompts / skills	`/api/prompt-repo*`, `/api/prompt-deployments`, `/api/skills*`, `/api/skills/serve/*`	`prompts.go`, `prompts_lifecycle.go`, `skills.go`, `skills_serving.go`

Group	Representative routes	Handler file(s)
Config / platform	/api/config, /api/proxy-config, /api/vector-store-config, /api/smtp-config, /api/cache, /api/plugins, /api/feature-flags, /api/cluster, /api/circuit-breaker/*, /api/load-balancer*	config.go, plugins.go, featureflags.go, cluster.go, circuitbreaker.go, cache.go
Enterprise / SCIM	/api/scim/*, /api/scim/oauth/*	scim.go, scim_handlers.go, scim_oauth.go, scim_auth.go
Browser AI	see §9	browser_ai*.go
WebSocket	/ws, WS tickets, responses/realtime WS	websocket.go, wsrealtime.go, wsresponses.go, ws_ticket.go
Dev	/api/dev/* + pprof (dev mode only)	devpprof*.go

Health and observability endpoints

- GET /health and GET /api/health ping configured ConfigStore, LogsStore and VectorStore services concurrently with a 10-second timeout. A healthy response reports status healthy and component checks; a dependency failure returns HTTP 503. The client.disable_db_pings_in_health option can disable these dependency pings, so a process-only health response must not be used as full readiness.
- GET /robots.txt — static Allow: /.
- GET /api/version — build version.
- Logging skip list (no request logging): /health, /_next, /api/dev.
- Metrics (Prometheus) and traces come from the telemetry/otel plugins and the Logs API.
- Dev builds additionally expose pprof handlers (gated by devpprof_prod.go in release builds).

Data model key tables

ConfigStore tables

- **`governance_users`** — `id, username(uniq), email, password, role(admin|sub_admin|user), status (pending|approved|rejected|email_unverified|disabled), budget, rate_limit, budget_id, rate_limit_id, allowed_prompt_repos, allowed_sections, reviewed_at, external_id, created_at, updated_at. IsApproved()` treats an empty status as approved (legacy rows).
- **`rbac_roles`** — `id, name(uniq), description, is_system_role, dac(all-data|own-data), permission_ids(JSON), created_at, updated_at`.
- **`access_profiles`** — `id, name(uniq), description, is_active, version, calendar_aligned, tags(JSON), spec(JSON), created_at, updated_at` (reusable governance policy templates with activate / deactivate / clone operations).
- **`governance_business_units`** — id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON).
- Grouped in tables/workspace.go: alert_channels, circuit_breaker_policies, mcp_tool_groups, prompt_deployments, workspace_settings, virtual_key_users, virtual_key_teams, virtual_key_customers, audit_logs.
- Plus the core governance tables: teams, customers, virtual keys, provider keys, models, model configs, pricing + pricing overrides, budgets, rate limits, routing rules, prompts/versions/sessions, skills, folders, feature flags, plugins, sessions, temp tokens, distributed locks, SMTP, vector-store config, MCP clients/library/OAuth2 issuance.

Browser AI tables

Table	Purpose	Notable columns
browser_ai_logs	one row per intercepted prompt / upload	id, timestamp, platform, domain(host), user_prompt_full, action, status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name/type/size/path/expiry, metadata(JSON)
browser_ai_search_logs	search-engine queries + clicked results	id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, url, host, agent_id, predictive_risk
browser_guard_rules	guard rules	id, name, pattern, rule_type(regex or ai_bot), severity, action, warning_message, active, description, bot_provider, bot_model, bot_prompt, bot_reference_image, bot_reference_image_type
browser_ai_target_websites	monitored AI sites	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED), parent_id
browser_ai_control_settings	single-row interaction policy (browser-controls-default)	upload/download warning text and policy toggles
browser_ai_agents	Guard fleet inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email(+pinned), uninstall_key_hash, uninstall_key_enc, has_uninstall_key, uninstall_key_rotated_at, proxy_bundle_sha
browser_ai_agent_settings	company-level uninstall key	id, uninstall_key_enc
browser_guard_fleet_config	company Guard defaults that agents pull on heartbeat	id, … notes
browser_guard_rebuild_logs	Guard package rebuild history	id, status(success/failed), bundle_sha, message, log, created_at
browser_ai_warning_emails	employee warning e-mail log	id, sent_at, to_email, message, agent_id, error_detail

- Attachment bytes stay on disk (APP_DIR/attachments); the DB stores size, path and expiry metadata.
- BrowserAIManager (same file) implements every CRUD/aggregation API used by the handlers,

including GuardSeverityScore (severity → predictive risk score + label) and NormalizeBrowserAIAgentType.

Browser AI Guard technical detail

Backend routes

Logs / insights

```
GET    /api/browser-ai/logs                list + filter prompt logs
```

```
GET    /api/browser-ai/logs/stats           aggregated stats
DELETE /api/browser-ai/logs                 delete all
POST   /api/browser-ai/logs/bulk-delete
DELETE /api/browser-ai/logs/{id}
GET    /api/browser-ai/search-logs
POST   /api/browser-ai/search-logs          agent ingest
DELETE /api/browser-ai/search-logs
POST   /api/browser-ai/search-logs/bulk-delete
DELETE /api/browser-ai/search-logs/{id}
GET    /api/browser-ai/insights/stats       Guard Insights
```

Rules / controls / targets

```
GET|POST       /api/browser-ai/rules            + POST /rules/import, PUT|DELETE /rules/{id}
POST           /api/browser-ai/rules/test-bot     test an AI Guard Bot rule
POST           /api/browser-ai/rules/generate-regex  generate regex from a policy description
GET|PUT        /api/browser-ai/controls           interaction controls (single row)
GET            /api/browser-ai/targets            + POST /targets, POST /targets/import,
                                                    PUT|DELETE /targets/{id}
```

Fleet / agents / setup

```
GET    /api/browser-ai/agents                      list agents (filters: status, source, search)
POST   /api/browser-ai/agents/heartbeat            agent registration + health (Guard secret)
POST   /api/browser-ai/agents/bulk-delete
DELETE /api/browser-ai/agents/{id}
PUT    /api/browser-ai/agents/{id}/contact-email
GET    /api/browser-ai/agents/settings
GET|PUT /api/browser-ai/agents/uninstall-key       company uninstall key
GET    /api/browser-ai/agents/{id}/uninstall-key
POST   /api/browser-ai/agents/{id}/uninstall-key/rotate
POST   /api/browser-ai/agents/uninstall-verify     employee key check
POST   /api/browser-ai/agents/uninstall            agent requests removal
POST   /api/browser-ai/agents/uninstall-ack
POST   /api/browser-ai/agents/uninstall-status
POST   /api/browser-ai/agents/{id}/remote-uninstall  admin-initiated removal
GET|PUT /api/browser-ai/fleet-config               company-wide Guard defaults
GET    /api/browser-ai/setup/info
GET    /api/browser-ai/setup/download.zip          aliases: -windows.zip, -mac.zip
POST   /api/browser-ai/setup/rebuild               rebuild + republish Guard packages
GET    /api/browser-ai/setup/rebuild-history
GET    /api/browser-ai/setup/proxy-bundle.json
GET    /api/browser-ai/setup/proxy-bundle.zip      hot-update bundle for installed Guards
POST   /api/browser-ai/send-warning-email
```

Ingest / PAC / helpers

```
POST /api/browser-ai/intercept          agent prompt/upload event
POST /api/browser-ai/intercept-file     agent file event
GET  /api/browser-ai/attachments/{id}   attachment download (auth-gated)
GET  /api/browser-ai/proxy.pac          PAC script
GET  /api/browser-ai/pac                PAC script (alias used by GPO / network proxy)
GET  /api/browser-ai/ollama-models      models available for AI Guard Bot rules
```

Guard fleet authentication no user session

`isPublicBrowserAIRoute` and `isGuardKeyBrowserAIRoute` (`handlers/middlewares.go`) let the Guard agent call the backend **without a dashboard session cookie**:

- **Public (secret-validated):** `intercept`, `intercept-file`, `search-logs` (POST), `proxy.pac`, `pac`, `agents/heartbeat`, `agents/uninstall-verify`, `agents/uninstall`, `agents/uninstall-ack`, `agents/uninstall-status`.
- **Guard-key:** `targets`, `rules`, `controls`, `fleet-config` (GET), `setup/download*.zip`, `setup/proxy-bundle.json`, `setup/proxy-bundle.zip`.
- The fleet secret is `GATEWAY_GUARD_SECRET`. With `GATEWAY_GUARD_REQUIRE_SECRET=1` (production default in `.env.example` and compose) agent APIs **fail closed** when the secret is missing or wrong (`browser_ai_guard_security.go`).
- `GATEWAY_PAC_ALLOW_QUERY_PROXY=0` (default) makes the PAC ignore a client-supplied `?proxy=` and use

only fleet/env values — anti PAC-hijack.

Guard desktop agent

`gateway_agent.py` is the entry point. Supporting modules:

Module	Responsibility
`guard_bootstrap.py`	single-instance lock, logging bootstrap, runtime version resolution
`agent_config.py`	env → JSON config precedence (`GATEWAY_BACKEND_URL`, `GATEWAY_PROXY_ADDR`, `GATEWAY_PAC_URL`, `GATEWAY_PAC_SYNC_SECONDS`, `GATEWAY_SERVER_MODE`, `GATEWAY_AGENT_TYPE`, `GATEWAY_LISTEN_HOST`, `GATEWAY_PAC_ADVERTISE_ADDR`, `GATEWAY_AGENT_ID`, `GATEWAY_AGENT_HOSTNAME`)
`agent_identity.py`	persistent agent ID + host/OS metadata for the fleet inventory
`agent_heartbeat.py`	periodic heartbeat, fleet config pull, remote-uninstall intent, health detail
`agent_health.py`	port checks (`port_open`, `free_proxy_port`), health loop
`agent_pac_content.py`	backend reachability check, PAC fetch, local PAC write
`agent_pac_server.py`	local PAC/health HTTP server; checks for an available configured listener when a conflict occurs
`agent_pac_orchestration.py`	apply PAC with cache-bust, clear/restore runtime, fail-open direct mode
`agent_proxy_engine.py`	runs the mitmproxy worker child process
`agent_proxy_bundle.py`	SHA-tracked hot-update bundle, mark-bad, relaunch, running SHA
`agent_certs.py`	installs the mitmproxy CA into the OS trust store, writes `ca_install_status.txt`
`agent_browser_policy.py`	browser QUIC / DoH policy adjustments
`agent_autoupdate.py`	compares the packaged agent release metadata and handles approved update bundles when configured
`agent_lifecycle.py`	first-run prompt, uninstall flow (key-gated), user messages
`agent_logging.py`	file logging, `get_resource_path`
`agent_http.py`, `agent_state.py`, `guard_platform.py`	HTTP client, local state, OS paths (`data_dir`, `log_hint_path`, `register_autostart`)

Default paths: data `%LOCALAPPDATA%\YP-Raksha\Guard` (Windows) / `~/Library/Application Support/YP-Raksha/Guard` (macOS); log `gateway_guard.log`; CA status `ca_install_status.txt`.

mitmproxy addon

The addon is **not** a normal Python package. `gateway_proxy_parts/MANIFEST.txt` lists the parts that are loaded **in order into one shared namespace** (preserving the original single-file monolith behaviour):

```
config_caches_rules.py      config, caches, target/rule fetch + matching
helpers_prompts.py          prompt extraction + helper utilities
uploads_detect.py           upload / file detection
file_policy.py              per-file-type policy decisions
extract_office_backend.py   Office/PDF/image text extraction + backend calls
responses_inject.py         response rewriting / block injection (SSE aware)
responses_addon.py          response learning (e.g. filename discovery)
```

Flow: target-domain match → extract prompt/attachment → evaluate rules (regex locally; `ai_bot` via the configured LLM/Ollama) → verdict `Allowed | Redacted | Warned | Blocked | SiteBlocked | Bot Answered` → inject the browser-visible response (JSON error, Anthropic SSE Guard policy-block event, etc.) → POST the event to `/api/browser-ai/intercept`.

Editing any part requires rebuilding the Guard (`installer/build_installer.bat` on Windows, `installer/build_macos.sh` on macOS) and keeping cross-part names intact.

Guard distribution and hot update

- Build: `make build-guard-windows` (Inno Setup) / `make build-guard-mac` (+ `-pkg`) — see `apps/browser-guard/README.md`. Without `make`, run `installer\build_installer.bat`.
- Config sync: `python apps/browser-guard/scripts/sync_config_from_env.py` regenerates `config/gateway_guard_config.json` and the docs from `.env` (`SERVER_DOMAIN`, `GATEWAY_PROXY_ADDR`, `PAC_HTTP_PORT`, `GATEWAY_GUARD_SECRET`).
- Server-side artifacts live in `apps/browser-guard/release/` and are copied into the image (`/app/release`), so **Browser AI → Setup → Download Setup ZIP** serves them.
- `Browser AI → Setup → Rebuild & Publish` re-publishes the proxy/agent Python sources as a bundle (`/api/browser-ai/setup/proxy-bundle.zip`); installed agents pick it up via `agent_proxy_bundle.py`.
- Version file: `apps/browser-guard/release/VERSION.txt` → `1.1.30`.

Frontend

Build and integration

- Vite + React 19 + TanStack Router (file-based routes under `ui/app/**`, generated into `ui/app/routeTree.gen.ts`).
- State: Redux Toolkit with RTK Query APIs under `ui/app/_fallbacks/enterprise/lib/store/apis/*` (accessProfileApi, alertChannelsApi, auditLogsApi, businessUnitsApi, circuitBreakerApi, clusterApi, connectorsApi, loadBalancerApi, mcpToolGroupsApi, promptDeploymentsApi, rbacApi, scimApi, virtualKeyUsersApi, …).
- `npm run build:gateway` performs the product rebrand/enterprise build; `npm run build` runs Vite, typecheck and copies the embedded UI into `transports/gateway-http/ui`, which ships with the Gateway server; no separate production web server is required.
- Dev server: `npm run dev`; `make dev` runs UI + API together.
- Quality gates: `npm run typecheck` (tsc --noEmit), `npm run lint` (oxlint), `npm run format` (oxfmt).

Pages

Top level: `/` (landing), `/login`, `/signup`, `/oauth/consent`, `/pprof`, `/workspace/**`, plus `__root`, `__notFound`, `__error`.

Workspace pages (one `page.tsx` per route, each with a `layout.tsx`):

Section	Routes
Observability	`dashboard`, `logs`, `logs/connectors`, `mcp-logs`, `observability`, `audit-logs`, `alert-channels`
Browser AI	`browser-ai` (tabs: `overview`, `logs`, `search-logs`, `rules`, `targets`, `agents`, `telemetry`, `setup`)

Section	Routes
Models	`providers`, `providers/model-limits`, `providers/routing-rules`, `model-catalog`, `model-limits`, `custom-pricing`, `custom-pricing/overrides`, `complexity-router`, `routing-rules`, `routing-rules/tree`, `circuit-breaker`
MCP Gateway	`mcp-gateway`, `mcp-registry`, `mcp-registry/library`, `mcp-registry/oauth-callback`, `mcp-tool-groups`, `mcp-sessions` (+ `auth`, `auth-failed`, `auth-success`), `oauth-grants`, `mcp-auth-config`, `mcp-settings`
Governance	`governance`, `governance/virtual-keys`, `governance/users`, `governance/teams`, `governance/business-units`, `governance/customers`, `governance/rbac`, `governance/access-profiles`, `rbac`, `virtual-keys`, `scim`, `scim/oauth-discover-callback`
Guardrails	`guardrails`, `guardrails/configuration`, `guardrails/providers`
Platform	`plugins`, `cluster`, `adaptive-routing` (+ `settings`), `prompt-repo` (+ `prompts`), `skills-repo`, `docs`, `config` (+ `api-keys`, `caching`, `client-settings`, `compatibility`, `feature-flags`, `logging`, `mcp-gateway`, `observability`, `performance-tuning`, `pricing-config`, `proxy`, `security`)

Sidebar and section gating

`ui/components/sidebar.tsx` builds the menu from `ui/lib/constants/workspaceSections.ts`:

```
observability | browser-ai | models | mcp-gateway | plugins | governance |
guardrails | cluster-config | adaptive-routing | prompt-repository | skills-repository | settings
```

Each section has a `defaultPath` and optional child `items`. Visibility is decided by:

1. RBAC hooks — `useRbac(RbacResource.X, RbacOperation.View)`.
2. Role checks for admin-only pages (Users requires `admin` or `sub_admin`).
3. Section grants for section-scoped sessions (`scopedSidebarSections` from `authStatus.allowed_sections`), with automatic redirect to `getDefaultPathForSections(...)` when the user lands on a route they are not granted.

Browser AI is reachable with `Logs` view RBAC **or** the `admin`/`sub_admin` role (`hasBrowserAiAccess`), matching the backend mapping of `/api/browser-ai/*` → `Logs`.

Browser AI UI implementation files

`ui/app/workspace/browser-ai/`: `page.tsx` (tabs, tables, dialogs), `browserAiTypes.ts`, `browserAiConstants.ts` (`GUARD_BOT_OLLAMA_PROVIDER = "ollama"`, `GUARD_BOT_OLLAMA_MODEL = "llama3.2"`, reference image max 512 KB), `browserAiFormat.ts`, `browserAiLogHelpers.ts`, `browserAiGuardHelpers.ts`, `guardBotModelPickers.tsx`, `guardRuleAIEvaluatorFields.tsx`, `guardRuleImportDialog.tsx`, `targetImportDialog.tsx`, `regexLiveTestPanel.tsx`, `logPromptPreviewCell.tsx`, `logTimestampCell.tsx`, `logBadges.tsx`, `attachmentPreview.ts`, `relatedHosts.ts`, plus import templates under `examples/`.

Types as implemented: `GuardRuleAction = "BLOCK" | "REDACT" | "WARN"`, `GuardRuleSeverity = "CRITICAL" | "HIGH" | "MEDIUM"`, `GuardRuleEvalMode = "ai" | "regex"`.

Security model

Area	Implementation
Dashboard sessions	Session rows stored in DB; `token` cookie with `HttpOnly` + `SameSite=Lax` (+`Secure` only for real TLS or a trusted proxy sending `X-Forwarded-Proto: https`); default 24 h TTL
WebSocket auth	Cookie-only (`/api/session/ws-ticket` issues short-lived tickets); legacy `?token=` query auth removed (token-leak risk)

Area	Implementation
Inference auth	Virtual Key or provider key via `Authorization: Bearer` / `x-api-key`; `enforce_auth_on_inference` flag
Brute force	Per-IP: 30 failed logins / 15 min window; per-username lockout rows in the DB; remaining minutes returned to the client
Password reset	`PASSWORD_RESET_SECRET` (min 32 chars) HMAC-signed token, 10 min expiry; 5 requests/hour/target + 10/hour/IP; OTP max 5 attempts; 2 min cooldown
Registration	Self-signup with e-mailed code (`email_unverified`) → admin approval (`pending` → `approved`) or `rejected`; SCIM can set `disabled`
Authorization	Two RBAC layers (resource×operation, section grants) + audit middleware recording mutating `/api/*` actions
Secrets at rest	`framework/encrypt` encryption for DB-stored secrets when `GATEWAY_ENCRYPTION_KEY` is set (provider keys, alert channels, uninstall keys)
Guard fleet	Shared `GATEWAY_GUARD_SECRET`, fail-closed with `GATEWAY_GUARD_REQUIRE_SECRET=1`; PAC hijack protection via `GATEWAY_PAC_ALLOW_QUERY_PROXY=0`
Guard uninstall	Company and per-agent uninstall keys, stored hashed + encrypted, rotatable, with an audit trail and remote-uninstall flow
Transport	TLS terminated at the edge/reverse proxy; `TRUST_PROXY_HEADERS=1` honours `X-Forwarded-Proto`/`X-Forwarded-For` **only** from trusted peers (`TRUSTED_PROXIES` required for public edges)
CORS	`client.allowed_origins` (from `SERVER_DOMAIN`); optional `ALLOW_LOCALHOST_CORS=1` for a local UI against a remote API
Headers	`SecurityHeadersMiddleware` applied to every response
Open bootstrap	When auth is disabled, unauthenticated admin is allowed only from loopback or with `ALLOW_OPEN_AUTH=1` (local bootstrap only)
Inference hardening	Provider headers normalised (`x-api-key` removed), password-as-Bearer removed, error sanitisation (`errorsanitizer.go`)
Destructive ops	Admin-only guards (`adminOnlyLogs`, `requireGuardAdmin`); attachment access privileged

Build test and developer workflow

Gateway

```
make setup-workspace     # go work use ./core ./framework ./plugins/... ./transports
make dev                 # UI (vite) + API with hot reload (air)
make build               # build UI then the Go binary
make run                 # build + run
make fmt ; make lint     # gofmt / linter

make test                # Gateway HTTP transport tests (gotestsum)
make test-core           # core provider tests (PROVIDER=openai TESTCASE=… PATTERN=…)
make test-framework      # framework tests
make test-plugins        # plugin tests
make test-http-transport # HTTP transport tests
make test-governance     # governance tests
make test-mcp            # MCP tests (TYPE=… TESTCASE=…)
make test-semantic-cache # semantic cache e2e
make test-all            # all of the above
make run-e2e ; make run-e2e-ui ; make run-e2e-headed ; make run-e2e-api
make test-integrations-py ; make test-integrations-ts
make run-provider-harness-test ; make run-cli-harness-test
```

Reports: `make generate-html-reports`; cleanup with `make clean-test-reports`.

UI

```
cd ui
npm ci
npm run dev             # Vite dev server
npm run typecheck       # tsc --noEmit
npm run lint            # oxlint
npm run build           # vite build + typecheck + copy into transports/gateway-http/ui
```

Guard

```
make sync-guard-config      # regenerate Guard config/docs from .env
make build-guard-windows    # Windows: Setup.exe + Windows ZIP (Inno Setup 6 required)
make build-guard-mac        # macOS: .app + macOS ZIP (must run on a Mac)
make build-guard-mac-pkg    # macOS: .pkg from the existing .app
make help-guard             # list Guard targets
```

Guard unit tests live in `apps/browser-guard/agent/tests/` and `apps/browser-guard/proxy/tests/` (PAC targets, proxy bundle, file-type extraction, latency, rule actions, target predict, upload names).

Docker Helm

```
make docker-image    # LOCAL=1 uses deploy/docker/Dockerfile.local (go workspace based)
make docker-run      # run the container (CONFIG=… optional)
make helm-index      # repackage chart + regenerate deploy/helm/index.yaml
```

Conventions

- Go modules resolve through `go.work`; add new modules with `go work use`.
- Handlers keep the `RegisterRoutes(router, middlewares...)` pattern and are grouped by feature.
- UI files use camelCase names, Radix primitives from `components/ui`, and RTK Query for API calls.
- Guard proxy parts must stay in `MANIFEST.txt` order.

Extension points

Goal	Where to work
Add an LLM provider	new package under `core/providers/<name>` implementing the provider contract, then register it in the provider registry
Add a pass-through API surface	new file in `transports/gateway-http/integrations/` + registration in `integrations/router.go`
Add a plugin	new module under `plugins/<name>` exposing `PluginName`; add a `case` in `loadBuiltinPlugin` (`server/plugins.go`) and add the module to `go.work`
Add a dashboard API	new `handlers/<feature>.go` with `RegisterRoutes`, then call it from `server.RegisterAPIRoutes`
Gate a new route with RBAC	extend `PathRequirementFor` (`framework/rbac/enforce.go`) and, for section-scoped sessions, `SectionRequirementFor` (`framework/rbac/sections.go`)
Add an RBAC resource	add it to `RBACResourceNames` (`framework/configstore/workspace.go`); permissions are generated from the cartesian catalog
Add a UI page	create `ui/app/workspace/<route>/page.tsx` + `layout.tsx`, add the section item in `ui/lib/constants/workspaceSections.ts`, and the nav entry in `ui/components/sidebar.tsx`
Add a Guard detection	edit the relevant part in `apps/browser-guard/proxy/gateway_proxy_parts/` (keep MANIFEST order) and rebuild the Guard

Goal	Where to work
Add a Guard rule type	`BrowserGuardRule` (`framework/logstore/browser_ai.go`) + the UI rule editor + the proxy evaluator
Change gateway guardrail behaviour	`plugins/guardrails` + `/api/guardrails/*`

Glossary

Term	Meaning in this codebase
Virtual Key (VK)	A gateway-issued credential that maps to budgets/limits/teams/customers and is used by inference clients
ConfigStore	Config/workspace database (users, roles, keys, providers, settings)
LogsStore	Log database (LLM logs, MCP logs, Browser AI logs, aggregates)
Section grant	Sidebar-section permission (`governance/users`) used for section-scoped sessions
DAC	Data access scope on a role: `all-data` or `own-data`
Target website	A monitored AI domain (with platform name, host role and monitored/blocked flags)
Guard rule	A detection rule: `regex` (pattern) or `ai_bot` (LLM-judged, optional reference image)
Guard agent	A Guard installation reporting to the fleet (`endpoint` or `network`)
Fleet config	Company-wide Guard defaults that agents pull on heartbeat
PAC	Proxy auto-config script served by the backend, pointing browsers at the Guard proxy
Proxy bundle	Server-published hot-update bundle of the Guard proxy/agent Python code
Uninstall key	Company/per-agent secret required to turn the Guard off
Intercept	The Guard → backend event that records a prompt, upload or verdict

Enterprise deployment context

This manual describes implementation behavior and the repository architecture. It is not a substitute for a bank-specific design, a release support matrix, security certification, performance benchmark or operating procedure. The companion On-Premises Deployment Architecture provides a separately labelled planning envelope for DC, DR and UAT at 120,000 managed endpoints.

120,000-endpoint fit: YP-Raksha is a suitable candidate for a 120,000-endpoint bank deployment when implemented against the companion reference architecture and accepted after release-specific load, resilience and endpoint qualification. The plan models 12,000 busy-period active users, 150 Browser AI Guard events/second and 200 inference requests/second with a full-capacity DC and a DR site reserved to scale to the same target. These are planning targets, not a benchmarked product limit or unconditional production guarantee.

For strict residency, keep the Gateway, relational stores, Browser AI logs, model services, collectors, backups, DNS, identity, SMTP relay and artifact repositories under bank control in India. The product also contains provider and connector integration paths that can send data externally when configured. Disable these or route them to internal services, then enforce default-deny egress at the network layer. Review the architecture document's data-flow map and verification evidence requirements before representing the deployment as no-egress.

The repository contains Windows and macOS Browser Guard packaging paths. It does not provide a complete versioned browser/VDI certification matrix in the source reviewed for this pack. Treat browser and VDI combinations as UAT targets until exact release-level compatibility evidence is recorded. Likewise, cluster functionality is not a cross-site DR guarantee; database replication, site promotion and failback are infrastructure and operations responsibilities.

Client deployment profile at a glance

Client requirement	YP-Raksha reference position	Detailed design
120,000 managed endpoints	Suitable candidate for qualification against the reference design. The planning case assumes 12,000 busy-period active users, 150 Browser AI Guard events/second and 200 inference requests/second. These are design targets, not measured product limits or an SLA.	Companion On-Premises Deployment Architecture, Sections 3–5; this manual, Sections 27–29.
DC, DR and UAT	DC is sized for the planned production load; DR reserves capacity to scale to the same target; UAT is isolated for release, endpoint, load and recovery qualification. Database/site failover and measured recovery objectives depend on the bank topology and exercise results.	Companion architecture, Sections 4–5 and 21; this manual, Sections 20–21 and 26.
Operating systems	Windows and macOS Guard packaging paths are present. Exact supported editions and versions must be confirmed against the selected signed release. No Linux endpoint Guard installer is documented; Linux may be used for Gateway infrastructure according to the bank platform standard.	This manual, Sections 20.6 and 26.
Browsers and VDI	Chromium-based Chrome/Edge and Firefox have candidate extension packaging paths. Exact browser builds and managed deployment behavior require qualification. Safari is not qualified in this reference. No Citrix, Horizon, Azure Virtual Desktop or other named VDI platform is certified here.	Companion architecture, Section 6; this manual, Sections 20.6, 26 and 29.
Data remains inside the bank and India	This is achievable as a deployment boundary when all model, storage, identity, artifact, email, observability, backup and support paths are internal or disabled; all sites and copies remain in India; and network/endpoint egress is default-deny. Product configuration alone cannot prevent an open network route from carrying data outside.	Companion architecture, Sections 2, 7–8 and 20; this manual, Sections 15, 17, 25–26 and 28.

The companion architecture document contains the detailed node and storage planning tables, site topology, network flow matrix and denied-egress diagram. Use this summary to locate the design; validate all compatibility and capacity statements with the selected release and the bank's own qualification evidence before treating them as supported production commitments.

Implementation boundaries relevant to product selection

Area	Repository evidence	Selection/deployment implication
Provider support	Provider adapter packages exist for multiple vendor families and operation types	Match required operations against the chosen adapter; do not assume equal feature coverage
Persistence	ConfigStore and LogsStore have PostgreSQL, MySQL and SQLite drivers	This pack uses PostgreSQL as the planning baseline; verify the release/database support matrix

Area	Repository evidence	Selection/deployment implication
Browser Guard endpoints	Windows and macOS packaging/build paths exist	Verify exact OS editions, versions, certificate deployment and endpoint management in UAT
Browser and VDI	Proxy/PAC and certificate mechanisms exist; a complete certification matrix is not present	Record exact tested browser/VDI versions; do not claim vendor certification without evidence
Data residency	Provider, telemetry and connector egress capabilities exist	Use internal endpoints and block public egress to establish the bank's residency boundary
DR	Application clustering and deployment tools exist	Design and exercise database/site replication, promotion, routing and failback separately
Performance	No benchmark result in this manual	Use the planning assumptions in the architecture document and qualify with a documented load test

Operational security review points

Before production, review the release-specific deployment with bank security and operations owners. At minimum, address privileged role provisioning, authentication/identity integration, secrets storage and rotation, database and backup encryption, TLS termination and trusted proxy configuration, audit access, payload logging, record retention, security alert routing, service account permissions, Guard agent update integrity, vulnerability scanning and emergency rollback.

Use internal artifact mirrors and review the actual release configuration. The repository's example deployment files contain public-service defaults; they are not evidence that a disconnected or no-egress deployment has been validated. The network owner should test the resulting system rather than relying on the intended application configuration.

Product architecture and request flows

YP-Raksha is organized as a Gateway service with an embedded administration UI, management APIs, inference APIs and optional integrations. The deployment has separate logical control and data planes, while model execution and selected storage/operations services can be separate bank-managed services.

End to end product architecture

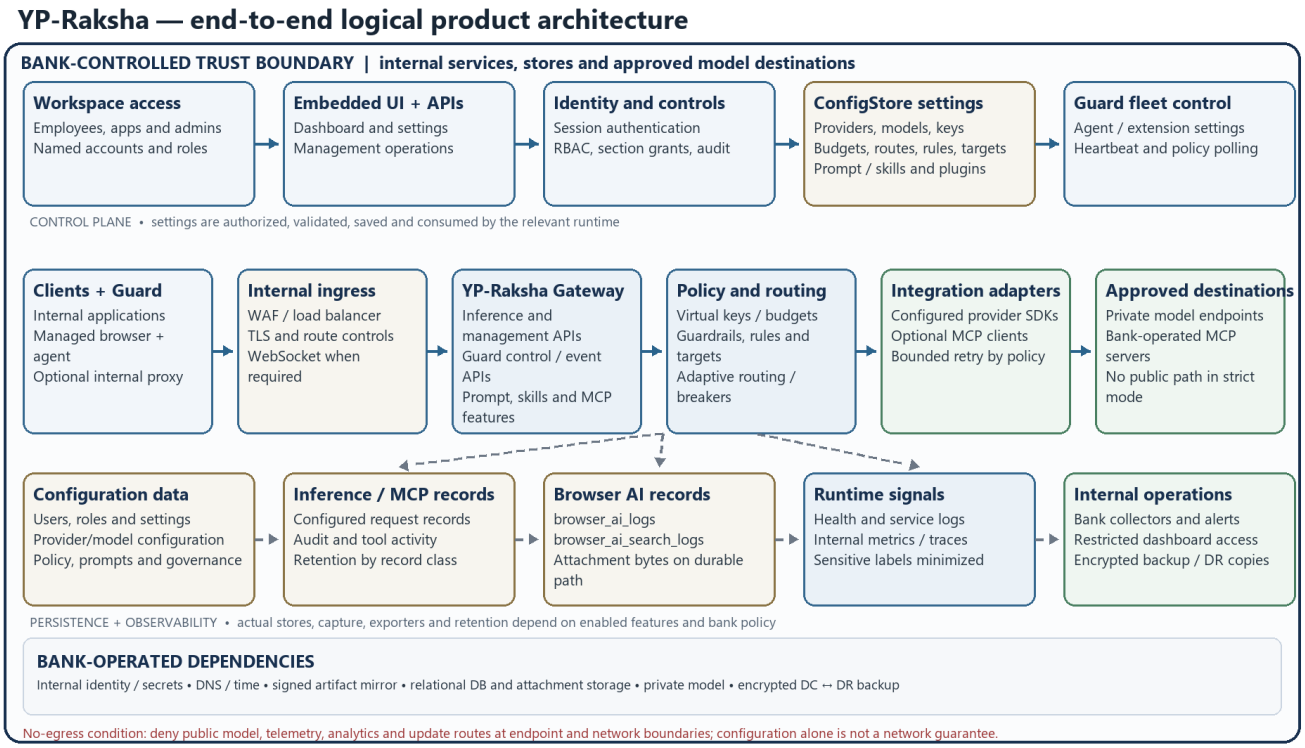


Figure 2. Consolidated YP-Raksha control plane, data plane, persistence and observability architecture. Optional components and data capture depend on configuration; bank network controls enforce the residency boundary.

This consolidated view connects employee/browser and application entry points with the Guard enforcement path, ingress, Gateway control and data planes, settings/policy persistence, internal provider destinations and observability. The diagram is a logical product view; site placement, node counts, allowed network rules and bank boundary proof are in the On-Premises Deployment Architecture.

Inference request lifecycle

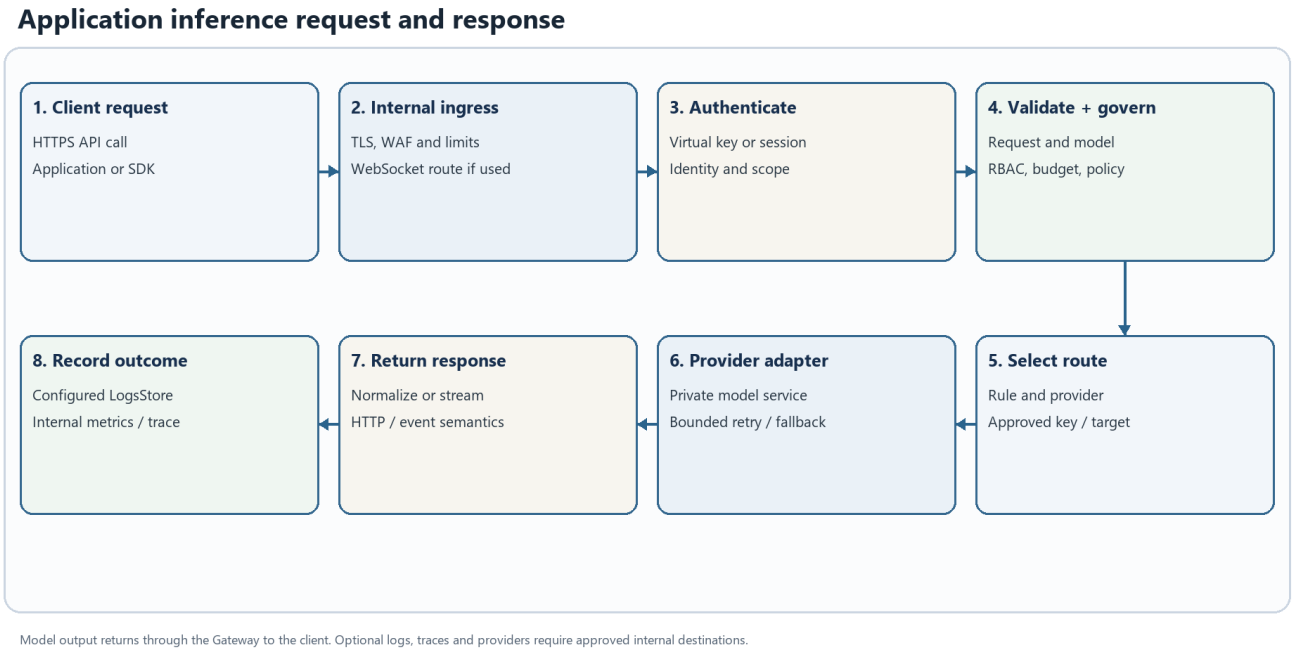


Figure 3. Application inference request and response. This is a logical workflow; enabled stages depend on deployment configuration.

1. The client submits an API request over HTTPS to the internal ingress. The route can be a standard inference endpoint or a supported provider/client-compatible integration.
2. The Gateway creates or accepts a request identifier, resolves credential/session context, authenticates the caller and applies configured virtual-key, role, budget and rate-limit checks.
3. Request validation and model/provider resolution run before provider execution. Missing or ambiguous provider/model data returns a client error; it is not treated as an upstream outage.
4. Policy hooks, compatibility conversion, routing rules, model selection and provider-key selection prepare the call. Enabled guardrail or cache plugins participate according to their configuration.
5. The selected provider adapter makes the outbound request to the configured destination. In a strict bank deployment this is a private bank-hosted or otherwise approved endpoint.
6. The core evaluates retry and key-rotation eligibility. If the call succeeds, the adapter normalizes the provider response or stream to the requested API shape.
7. Configured post-hooks run and response metadata is returned to the caller. Once a stream begins, a later failure cannot change the already-sent HTTP status.
8. Logging, tracing and metrics follow enabled configuration. Payload capture is separately controlled and must be minimized for sensitive data.

This is a logical sequence, not an unconditional feature promise. Provider/model support, API shapes, retry safety, guardrail failure policy and logging behavior are release- and configuration-specific.

Browser AI Guard decision lifecycle

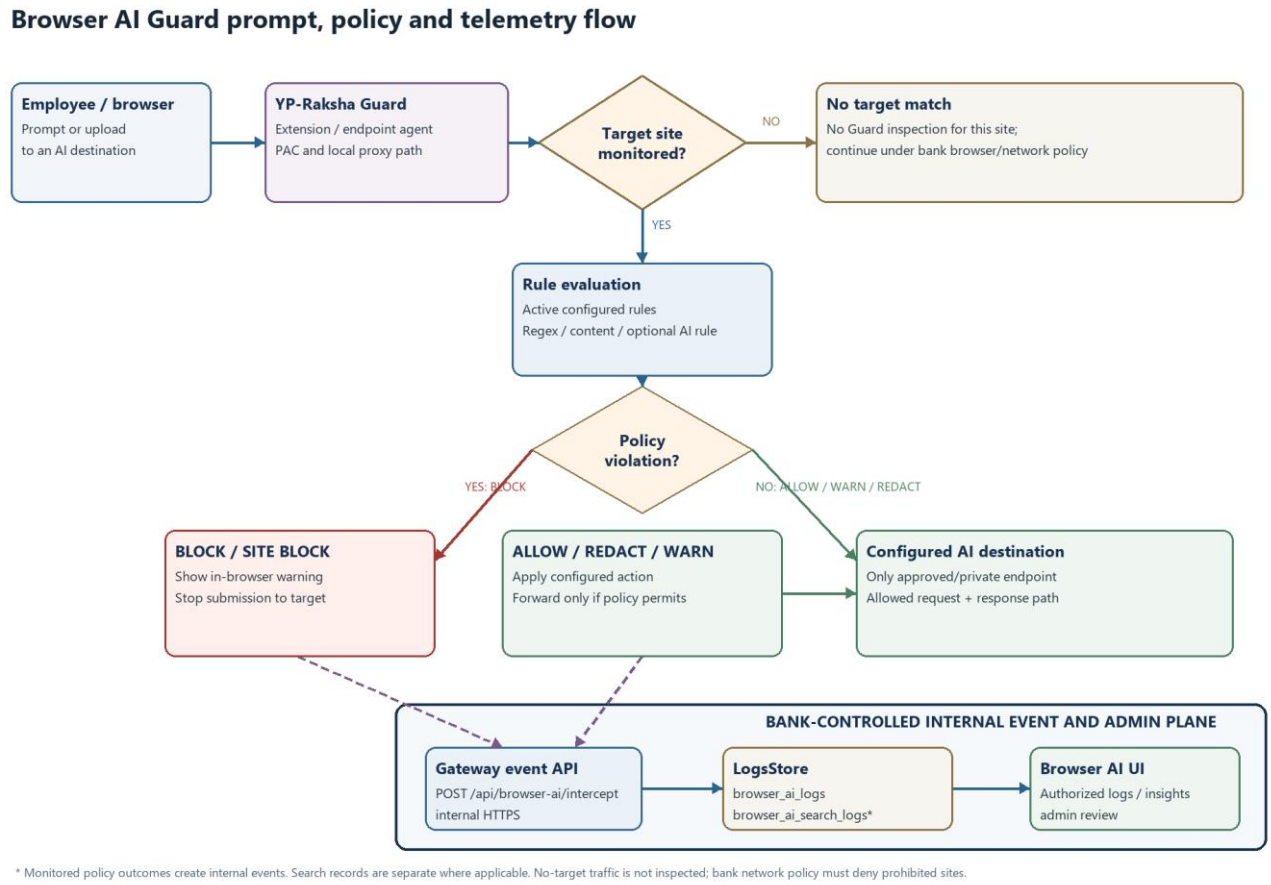


Figure 4. Browser AI Guard request, verdict and internal event flow. Exact behavior depends on configured rules and the validated release.

1. A managed browser requests a configured AI destination. Endpoint policy/PAC directs selected traffic to the local Guard proxy or an approved internal shared proxy.
2. The proxy checks target configuration, identifies the relevant request and extracts supported prompt or file content for policy evaluation.
3. Local regex rules are evaluated. If an AI Guard Bot rule is configured, the configured model/service may also be called; that destination must be approved and private for no-egress use.
4. The policy engine selects the configured action: allow, redact, warn, block, site-block or bot response. Matching order, rule scope and unmatched-site behavior must be acceptance-tested for the release.
5. Only traffic that policy allows to proceed is forwarded to the target. Redaction changes request content according to the configured rule; block/site-block prevents the upstream request. Warn behavior follows the configured control policy.
6. The response is passed back or transformed according to the proxy implementation. Streaming formats use provider-specific response handling; validate required providers and browser versions.
7. The Guard posts the policy event to the internal Browser AI API. The Gateway records it in LogsStore; permitted administrators review it in the embedded UI.
8. If the Gateway or event API is unreachable, do not assume this meets a bank fail-closed requirement. Combine endpoint settings with network rules that deny direct access to unapproved AI sites.

The exact content captured depends on traffic type and extraction support. Configured regex rules can express approved sensitive-term or pattern checks for DLP-style policy; the reviewed source does not establish a separate certified DLP engine. Treat prompts, file text, filenames, URLs and user identifiers as sensitive; configure collection, access, retention and deletion with the bank data owner.

Administration UI and policy change lifecycle

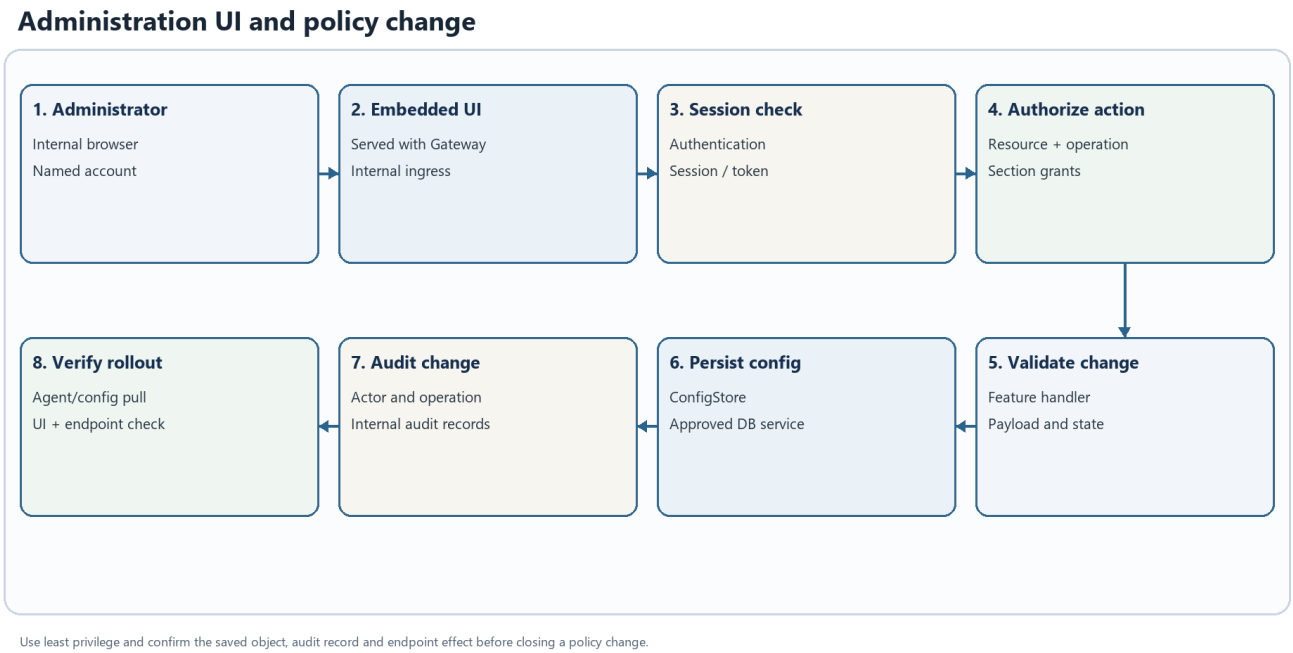


Figure 5. Administration UI and policy change. This is a logical workflow; enabled stages depend on deployment configuration.

1. An administrator opens the UI served with the Gateway application through the bank ingress; a separate public UI host is not required by the product build.
2. The administrator signs in through the configured authentication path. The browser holds a session credential according to application cookie/session configuration.
3. Each management API request passes authentication, audit middleware for mutating routes, resource-operation RBAC and applicable section-grant checks.
4. The feature handler validates the payload and requested operation. Invalid input returns a client error; a role denial returns an authorization error.
5. A valid update is written to the relevant ConfigStore model. If the action also changes grants or dependent objects, verify the full API result and audit event.
6. Mutating operations are added to the audit path. Operational logs and metrics are separate from business audit records and have separate access/retention needs.
7. Guard agents obtain fleet settings, rules and targets from internal Browser AI APIs during configured polling/heartbeat. Validate propagation time and endpoint identity in UAT.
8. The administrator verifies the resulting object, audit entry, service health and representative endpoint behavior.

Use named accounts and least privilege. The admin role is broad by design; scope other roles by resource, operation and section. Review configuration changes under the bank change process.

Optional MCP tool lifecycle

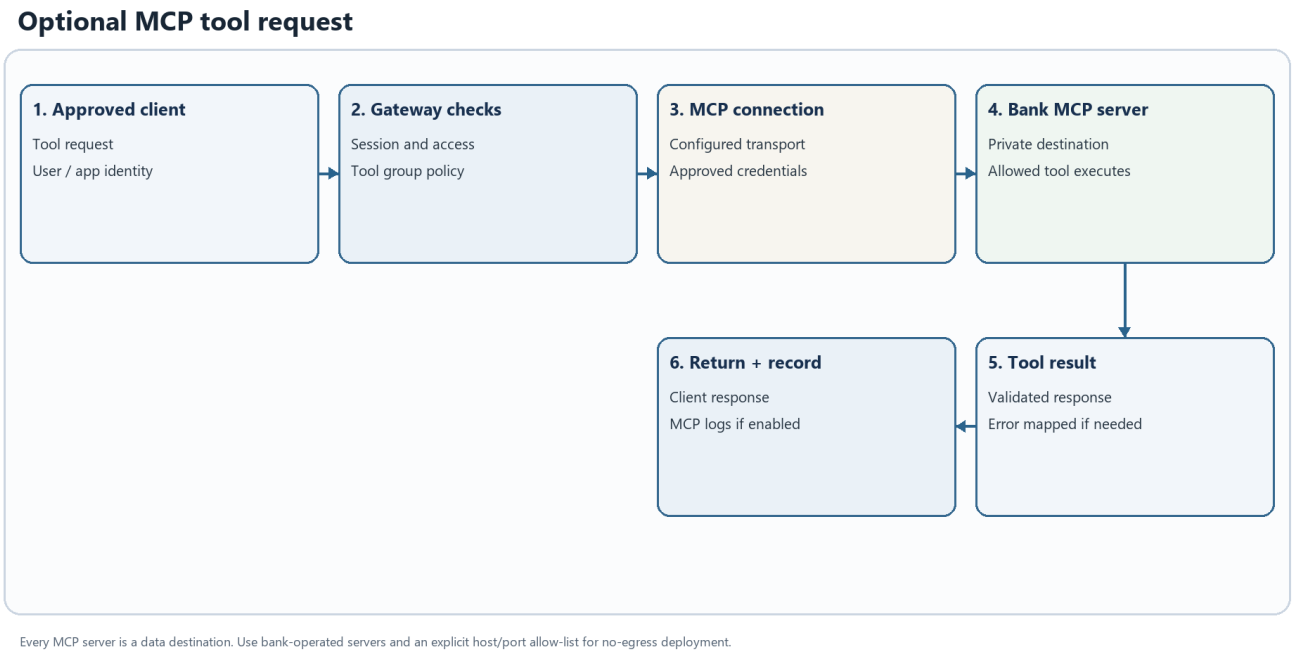


Figure 6. Optional MCP tool request. This is a logical workflow; enabled stages depend on deployment configuration.

1. An approved client request selects an enabled MCP client, session and tool group.
2. The Gateway checks the requesting identity, tool permissions and configured credentials.
3. The MCP client connects to the configured server using its selected transport and authentication method.
4. The remote tool executes and returns a result to the Gateway.
5. The Gateway returns the result to the client and records MCP usage/error information according to logging configuration.

Every MCP server is a potential data destination. For no-egress use, allow only bank-operated MCP servers, limit reachable hosts and ports, and review tool inputs, outputs, credentials and logs.

Observability and settings to policy propagation

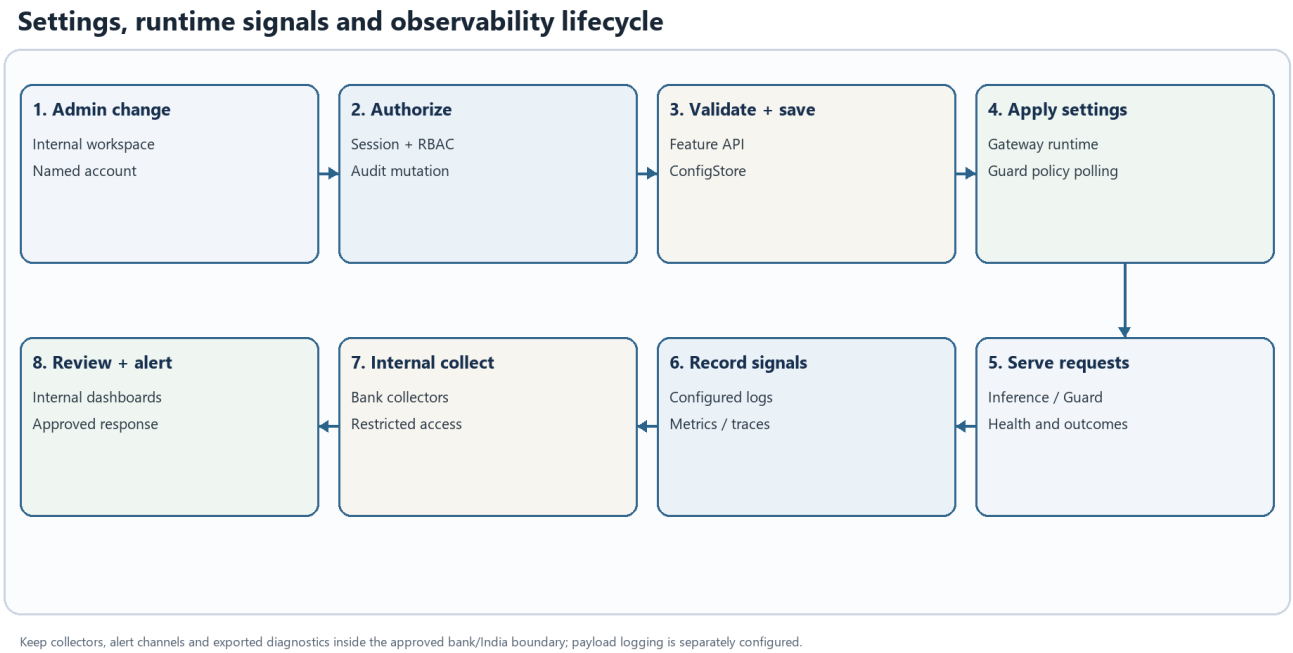


Figure 7. Settings, runtime signals and observability lifecycle. This is a logical workflow; enabled stages depend on deployment configuration.

Configuration changes are authenticated and authorized through the management path, validated by the feature handler, persisted to ConfigStore and then consumed by the Gateway runtime or Guard polling path where applicable. Runtime health, request outcomes, events and enabled metrics/traces are written to their configured internal stores or collectors. The database, collector and alert destination must be bank-operated for the no-egress design. Not every request payload is logged; capture behavior is controlled separately and must be verified for the selected release.

State scaling and component ownership

Component	Runtime responsibility	Availability and recovery responsibility
Gateway replicas	HTTP APIs, inference orchestration, policy integration, UI and optional background features	Platform team: replica placement, ingress, resource limits, rolling updates and health probes
ConfigStore	Users, roles, settings, providers, keys, routing and operational configuration	DBA/platform: HA, TLS, connection limits, backups, restore and site replication
LogsStore	Inference, audit, MCP, Browser AI event and aggregation records	DBA/data owner: retention, index maintenance, archive, access and recovery
Attachment and release files	Browser AI attachment bytes and Guard/package artifacts on configured paths	Platform team: persistent/shared volume behavior, permissions, capacity, backup and retention
Private model endpoints	Model execution and model-side queueing	Model owner: locality, authorization, capacity, quotas and availability
Internal observability	Metrics, application logs and trace receivers where enabled	Operations/security: internal-only endpoints, sensitive-label review, retention and alerts
Guard fleet	Endpoint proxy/PAC, local health, configuration exchange and event delivery	Endpoint/network teams: rollout, certificates, local data path, proxy policy, update and recovery

The relational database supports several drivers in the implementation. PostgreSQL is the reference production planning choice in this pack; qualify a release/database combination with the product owner and bank DBA. Optional cache, vector and object services are not all required for every deployment.

Error handling retries and recovery

Error response contract

The HTTP layer returns a JSON Gateway error object for the standard API error path. Fields are optional by error class; this is an illustrative shape, not a fixed response for every route.

```
{
  "is_gateway_error": false,
  "status_code": 429,
  "error": {
    "type": "rate_limit_error",
    "code": "rate_limit_error",
    "message": "Request limit reached",
    "event_id": "optional-event-id"
  },
  "extra_fields": {
    "provider": "configured-provider",
    "request_type": "configured-operation"
  }
}
```

An incoming x-request-id can be used as the request identifier; if absent the request context creates one. Preserve the identifier, timestamp, route, deployment/site, provider/model label, HTTP status, retry count and client application name when opening an incident. Do not include prompts, credentials or full payloads in an incident ticket unless approved.

The HTTP error path sanitizes selected internal error details before returning them to the client. Stack traces, SQL/database diagnostics and HTML/status-page bodies are hidden behind a client-safe internal message. Operational logs remain the diagnostic source; restrict them because they may contain sensitive request context.

Common failure classes and caller action

HTTP status / class	Typical meaning	Caller response	Operator check
400 Bad Request	Invalid JSON/parameters, missing provider/model, unsupported operation/stream combination	Correct the request; do not retry unchanged input	Validate schema, route and model selection
401 Unauthorized	Missing/invalid client credential, expired session, or provider credential error	Re-authenticate or use a valid virtual key; do not retry a known bad key indefinitely	Check caller identity, key state, provider credential and session configuration
403 Forbidden	Caller lacks application/role permission, or provider key/account is denied	Request the required grant or use an authorized account	Inspect RBAC/section grants, virtual-key scope and provider permissions
404 Not Found	Unknown Gateway route/resource, model or object identifier	Verify endpoint, ID and API version	Confirm route registration, object lifecycle and model catalog
409 Conflict	Resource/state conflict on operations that use conflict semantics	Refresh resource state and retry only after resolving conflict	Inspect concurrent changes, duplicate operation and audit event
429 Too Many Requests	Gateway quota/rate limit, Guard uninstall lockout, or provider throttling	Back off; honor returned retry guidance; avoid burst retry loops	Separate caller budget from provider quota and inspect key attempts
500 Internal Server Error	Application, persistence or feature-handler failure	Retry only if operation is safe and cause is transient; retain request ID	Check service logs, DB connectivity, schema/configuration and plugin startup

HTTP status / class	Typical meaning	Caller response	Operator check
502 Bad Gateway	Provider/network failure, exhausted upstream credentials, or upstream response failure	Use bounded retry only where request semantics permit; do not assume write completion	Check private endpoint, TLS, route, provider status and key pool
503 Service Unavailable	Required store/service unavailable, no eligible key, Guard secret missing or health dependency failure	Retry after recovery with jitter; do not bypass policy	Check readiness dependencies, DB/vector store, secret injection and route eligibility
504 Gateway Timeout / deadline	Upstream or downstream operation exceeded its configured deadline	Treat write/submit completion as unknown until reconciled	Check request/ingress timeout, provider latency and DB wait
WebSocket/realtime error event	Route-specific handshake, session or provider event failure	Follow event type/code and approved reconnect policy	Check ingress upgrade headers, session/ticket, allowed origins and provider support

This is a diagnostic map, not a universal one-to-one status contract. Handlers and provider adapters can return different codes/messages; inspect the selected release API schema and acceptance results before client integration.

Retry key rotation and fallback behavior

The core retry loop uses configured MaxRetries and network backoff values. Backoff is exponential with jitter and bounded by its configured maximum. Current source classifies failures as follows:

Failure	Core behavior	Client/application design
Network/request transport error	Eligible for bounded retry; retain the same key because a new credential does not repair the path	Set finite timeouts; prevent caller and Gateway retries from multiplying without limit
Upstream 500, 502, 503 or 504	Eligible for bounded retry using the same key	Ensure the total deadline covers attempts and backoff
Upstream 401, 402 or 403	Eligible for key rotation when a provider key pool exists; failed credential is marked dead for that request	Correct or remove invalid keys; monitor credential exhaustion
Upstream 429 / rate-limit signal	Eligible for retry and key rotation where possible; a rate-limited key may be reconsidered after pool exhaustion	Respect quotas; avoid synchronized retries across replicas
Request-bound 4xx such as invalid input	Not retried by the core retry classifier	Fix the request; changing keys does not correct malformed input
Gateway internal error / request cancellation	Retry loop stops	Diagnose locally or cancel; do not automatically replay non-idempotent calls
Streaming error found before first usable chunk	Can be classified and retried before a stream is returned	Validate per provider; never assume a retry after content has been sent
Configured circuit breaker / fallback	Separate routing control can select an approved fallback when policy conditions match	Fallback is not automatic for every error; keep targets within approved locality/data class

A transport timeout can happen after an upstream service accepted the request. For non-idempotent operations such as file, image, video, batch or tool creation, completion may be unknown. Use operation IDs, provider status lookup, deduplication/idempotency keys where supported, and reconciliation before resubmission. The Gateway does not make every operation exactly-once.

Streaming WebSocket and partial response errors

Before headers/body are committed, a handler can return a normal HTTP error. After SSE/WebSocket/realtime output begins, the HTTP status cannot be replaced. A provider failure mid-stream can appear as a protocol error event, stream termination or incomplete response. Clients should:

- treat an incomplete stream as failed, retain the request ID and stop presenting partial output as a completed answer;
- close the stream and follow a bounded, application-approved reconnect policy;
- avoid replaying side-effecting tool calls without checking provider/tool state;
- align ingress idle/read timeouts with approved stream lifetime;
- verify SSE, WebSocket and realtime behavior separately in UAT for every required provider.

Operational recovery matrix

Symptom	Likely failure domain	First checks	Recovery path
/health returns 503	ConfigStore, LogsStore or configured VectorStore ping failure	Service DNS, port, TLS, credentials, pool exhaustion and DB status	Restore dependency; verify health and a read-only API; do not substitute disabled checks for recovery
All inference returns 400	Client request or model/provider resolution	Request schema, model name, provider config and operation	Correct payload or add the approved model/provider route
Inference returns 502/503	Provider endpoint, credentials, route policy or no eligible provider key	Gateway DNS/TLS/connectivity, provider health, key pool, virtual-key grants and egress denies	Restore private route or approved key; reduce traffic or use a pre-approved fallback
Repeated 429s	Caller quota, budget/rate policy or provider quota	Compare Gateway usage limits with provider response and per-key attempt trail	Apply bounded jitter, rebalance approved keys or request capacity; do not raise quota without owner approval
Guard heartbeat rejected or 503	Guard secret absent/mismatched, wrong backend URL, certificate or route	GATEWAY_GUARD_SECRET, GATEWAY_GUARD_REQUIRE_SECRET, agent version, URL, DNS/TLS and firewall	Correct via approved secrets and endpoint rollout; rotate if exposed
Prompt is not blocked	PAC bypass, target mismatch, rule scope/order or unsupported extraction	PAC result, active target, rule status, agent health, local proxy logs and direct egress path	Correct target/rule and retest; deny public AI at network edge for protected users
Browser works but event is missing	Event API reachability, secret, LogsStore failure or retention/filter	Agent delivery status, health, Browser AI API and time/filter	Restore internal API/store connectivity; confirm ingest and retention result
Uploaded attachment unavailable	Attachment path permission, expiry, volume or metadata/DB mismatch	Persistent path, free space, owner/mode, expiry and pod placement	Restore approved persistent storage; reconcile expired/orphaned metadata; retest access
Admin update returns 403	Session, role/resource permission or section grant	Identity, role, route requirement and audit log	Grant least-privilege access through authorized administrator
UI works but APIs fail	Ingress rewrite, cookie/origin, session expiry or API health	Browser network response, ingress route, TLS/cookies and Gateway log	Correct routing/session policy; verify a read-only API before retrying a mutation
WebSocket/realtime handshake fails	Ingress upgrade, short-lived ticket, auth/origin or provider support	Upgrade forwarding, ticket freshness, origin, route and provider	Fix ingress/auth and validate with a new ticket
500 after configuration change	Invalid dependent config, DB write/migration or plugin issue	Request ID, application log, audit entry, DB health and startup/plugin status	Roll back the specific change via approved method; preserve audit history

Symptom	Likely failure domain	First checks	Recovery path
No metrics/traces	Scrape/token, plugin/exporter or network rule	Metrics reachability, GATEWAY_METRICS_TOKEN, internal collector and egress policy	Correct internal scrape/export; do not enable a public collector as workaround

Incident evidence and escalation

Capture UTC timestamp; deployment/site and release digest; request ID or agent ID; route and operation; status/event type; provider/model label without credentials; retry count and attempt outcome; health/dependency status; matching log/trace IDs; recent approved change; and sanitized network error. Protect raw request/response content and attachments under bank data policy. Escalate product defects with a minimal reproducible request using synthetic content and exact release identifier.

Error handling decision flow

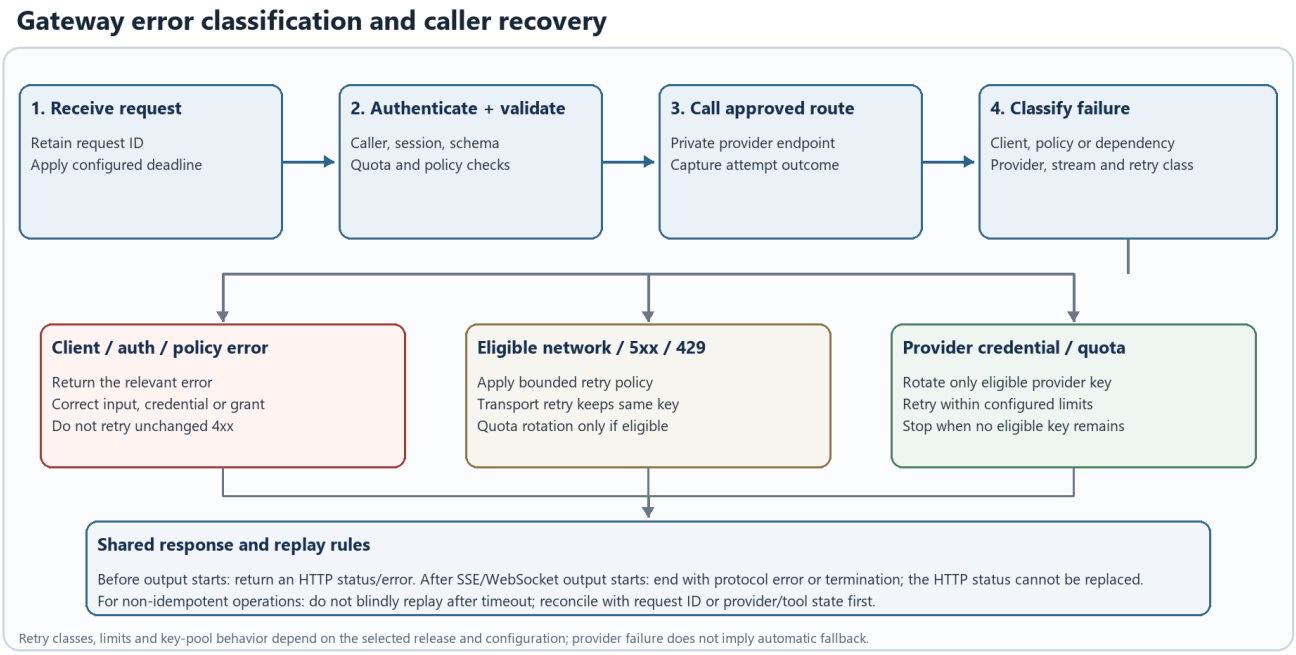


Figure 8. Gateway error classification and caller recovery. Branches are selected by the classified failure; enabled behavior depends on release configuration.

The runtime classifies client/auth/policy failures separately from provider and dependency failures. It retries only errors eligible under the configured bounded retry policy; credential rotation applies only to eligible provider-auth/quota failures and an available key pool. Once streaming output has started, a later failure is an incomplete stream or protocol error, not a replacement HTTP response. Callers must avoid blind replay of non-idempotent operations.

Deployment prerequisites and configuration

Required platform dependencies

Dependency	Required planning decision
Runtime platform	Select bank container/Kubernetes version, node OS/CPU architecture, ingress class and support policy; verify the release image

Dependency	Required planning decision
Internal artifact supply chain	Mirror approved signed Gateway image, Guard installer/bundle, chart and model/config assets; scan and record digests
Database	Provide HA relational storage supported by the release; use PostgreSQL as the reference baseline, then validate version, TLS, pool, backup and migration
Storage	Provide durable capacity for database, configured attachment directory and generated/release artifacts that must survive pod replacement
Network	Provide internal DNS/NTP, ingress, private model routes, database routes, endpoint HTTPS, observability and encrypted DC-to-DR replication
Identity/certificates	Decide authentication, SSO/SCIM/AD if used, TLS termination, trusted proxy CIDRs and CA trust distribution
Secrets	Use bank secrets manager or encrypted Kubernetes Secrets; define rotation/recovery for DB, encryption, bootstrap admin and Guard secrets
Operations	Define internal logs/metrics/traces, metrics access, alert routes, retention and incident owner
Endpoint rollout	Define Windows/macOS package deployment, certificate/PAC policy, agent identity persistence, VDI treatment and rollback
Residency	Approve provider/model, SMTP, licensing, update, connector and telemetry destinations; deny all other egress at network layer

Runtime configuration groups

The source .env.example is a template with development-oriented values. Inject production secrets through the approved secret manager.

Configuration group	Representative current keys	Purpose and handling
HTTP/runtime	APP_PORT, APP_HOST, APP_DIR, LOG_LEVEL, LOG_STYLE	Bind intended interfaces; persist APP_DIR data where local attachments/artifacts must survive
Database	DB_TYPE, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_MODE	Separate credentials by environment; require database TLS where supported
Secret encryption	GATEWAY_ENCRYPTION_KEY	Keep stable and protected under dual control; losing it may make encrypted stored credentials unrecoverable
Guard fleet	GATEWAY_GUARD_SECRET, GATEWAY_GUARD_REQUIRE_SECRET, GATEWAY_BACKEND_URL, GATEWAY_PROXY_ADDR, GATEWAY_AGENT_TYPE	Require strong secret in production; distribute securely and rotate through an endpoint rollout
PAC behavior	GATEWAY_PAC_ALLOW_QUERY_PROXY, GATEWAY_PAC_URL, GATEWAY_PAC_SYNC_SECONDS, GATEWAY_PAC_ADVERTISE_ADDR	Keep client override disabled unless approved; validate proxy address from endpoint network
Admin and mail	ADMIN_EMAIL, ADMIN_PASSWORD, SMTP_*, PASSWORD_RESET_SECRET	Change bootstrap credentials; use internal relay or disable mail; protect reset signing secret
Metrics/proxy trust	GATEWAY_METRICS_TOKEN, TRUST_PROXY_HEADERS, TRUSTED_PROXIES	Restrict metrics to internal scrapers; trust forwarded headers only from ingress peers
Branding	GATEWAY_COMPANY_NAME, GATEWAY_COMPANY_LOGO and product/footer settings	Set approved company/product/footer copy; validate landing, login and footer before release

Use only names present in the selected release's .env.example, chart and configuration loader. Do not copy sample passwords, public SMTP/DNS or public URLs into production.

Production packaging cautions

- Root Compose is a development/evaluation starting point. The current example includes public DNS resolvers and runs the application as root. Do not deploy it unchanged for bank production/no-egress.
- The optional Compose network-proxy profile is a lab configuration; permissive interception/TLS settings are not a production baseline.
- The Helm chart and values include public image/catalog examples. Confirm the chart version maps to the approved YP-Raksha release, mirror it internally, pin immutable digests and remove public endpoint defaults.
- Supply credentials through secret references or the bank secret integration. Do not put cleartext credentials in values files, environment literals, command lines or tickets.
- Use internal TLS ingress, database endpoints, signed artifacts and restricted service accounts. Keep profiling and development routes unavailable in production.
- UAT requires its own database, secrets, admin identities, model endpoint and telemetry. Do not restore production records into UAT without approved masking.

Docker and container deployment procedure

Use Docker Compose for an isolated evaluation or UAT smoke deployment only. It is not the reference runtime for the 120,000-endpoint DC design. Production should use the bank's supported Kubernetes/container platform with multiple Gateway replicas, durable storage, health probes, disruption protection, autoscaling limits and an internal ingress. The root Compose file is a developer starting point: it runs as root, includes public DNS resolvers, binds a host listener directly and has an optional proxy profile with permissive interception settings. Do not deploy that file unchanged in a bank environment.

The Compose Gateway service expects an externally managed database; the file does not create a production PostgreSQL HA service. It also publishes the Guard release directory and uses a persistent data directory. Keep application data on bank-managed durable storage, restrict that storage to the service identity, and confirm that container replacement does not remove attachments or generated artifacts.

The deployment sequence is:

1. **Import a release.** Obtain the approved signed image and chart from the product release channel, scan them, record their digests, and transfer them through the bank's controlled import process into an internal registry. Production nodes must not build from source or pull public images at runtime.
2. **Prepare the platform.** For production, create the namespace, service account, ingress, persistent volumes, network policies, resource requests/limits, readiness/liveness probes, disruption policy and replica/scaling limits from the release-matched manifest. Keep profiling and development routes disabled.
3. **Prepare configuration.** Create a bank-controlled environment file or secret-manager references using only the keys supported by the selected release. Keep the actual values in the bank's protected configuration store. Do not place credentials in a checked-in values file, command history, ticket, shared client document or support bundle.
4. **Prepare persistence.** Connect only to the provisioned bank database and durable application-data volume. Confirm database TLS, connection limits, retention, backup and recovery before starting the Gateway.
5. **Start UAT.** For a single-host Compose smoke test, validate the local Compose configuration without printing resolved values, then start only the Gateway service. Do not enable the optional shared network-proxy profile until it has an independently reviewed hardened design.
6. **Check readiness.** Confirm container readiness, application health, database read/write, authenticated UI, internal model access, internal telemetry and Guard event flow. Check logs locally and redact any prompt, token, host or credential values before sharing diagnostics.

7. **Promote.** Promote the same immutable image and release-matched manifest from UAT into DR and DC. Change only protected environment configuration; do not rebuild the image between environments.

Illustrative local Compose commands for an approved UAT host are:

```
docker compose config --quiet
docker compose up --detach gateway_tech
docker compose ps
docker compose logs --tail=200 gateway_tech
```

The host's `.env` must already be provisioned through the bank's protected process before running these commands. The resolved configuration contains sensitive connection values; do not save or attach a rendered Compose configuration to the client pack. For Kubernetes, apply the approved release manifest or chart from the internal registry using the protected values integration, then wait for the Gateway workload and its readiness checks to complete before routing traffic.

Database provisioning and lifecycle

PostgreSQL is the reference production database for this design. The codebase also contains MySQL driver paths, but a bank must verify the exact engine and version against the selected YP-Raksha release before using them. SQLite is not an appropriate HA store for the 120,000-endpoint design. Database version numbers, hostnames, listener values, database identifiers and credentials are deployment-specific and belong in the bank's secured DBA configuration.

Database concern	Provisioning requirement
Environment isolation	Provide independent UAT, DC and DR database services or approved isolated database contexts. UAT must not share production credentials or accept production writes.
Availability	Use a bank-operated HA topology across failure domains. Document promotion, fencing, client reconnection and failback; do not treat application clustering as database HA.
Roles and privileges	Use a dedicated service identity with only the runtime privileges required by the release. If schema migration needs DDL privileges, provide a controlled migration identity or DBA-run migration step and remove temporary privileges afterwards where supported.
Transport and access	Require database TLS, certificate validation, network allow-listing from Gateway workloads only, secret-manager delivery and credential rotation. Do not expose database services to endpoint or user networks.
Pooling and performance	Set per-replica pool limits so the sum of all Gateway replicas, jobs and operators remains within the database connection budget. Measure write latency, lock waits, pool saturation, WAL growth and replication lag at planned load.
Schema changes	Back up before release changes. Review the release migration notes and startup migration behavior; test migrations against a restored UAT copy. Do not assume a downgrade is safe.
Durability and recovery	Configure encrypted backups, transaction-log/WAL archiving where supported, point-in-time recovery, integrity checks, restore testing and India-resident DR copies. Keep encryption and secret-recovery material available under the bank's separate key process.
Data and retention	Apply retention and deletion schedules to inference, audit, Browser AI and operational records. If prompt or file capture is enabled, measure actual payload sizes and include attachment storage, indexes, transaction logs, replicas and backup copies in capacity.

ConfigStore and LogsStore hold different logical record classes. The physical database/schema split is a release and DBA design choice; follow the release's supported topology and make the access, backup, retention and recovery scope explicit. Browser AI attachment bytes are stored under the configured application data directory while related metadata is stored in the database. Back up and restore those components as a coordinated set, and verify attachment permissions and retention deletion after restore.

Browser Guard agent and extension installation

The desktop Guard agent and browser extension are separate components and both must be installed and configured for the selected protection flow. A portable Windows executable does not install the browser extension by itself. The release bundle contains a Windows agent (`YP_Raksha.exe`), a macOS package/archive (`YP_Raksha_macOS.zip`), and separate Chromium and Firefox extension packages. Exact filenames, signatures, extension identifiers, OS support and certificate behavior must be recorded against the selected release digest before rollout.

Endpoint component	Deployment method	Main installation and validation steps
Windows Guard agent	Distribute the approved release bundle through the bank's endpoint-management system (for example, its software distribution or device-management service). Extract the complete package into a protected application directory and run `YP_Raksha.exe` in the intended user context.	Verify the release signature/digest, endpoint configuration points only to the internal Gateway, autostart is registered as expected, the user can see Guard status, the agent reaches the internal control plane and the dashboard shows a healthy endpoint. Confirm the package's actual install and update behavior in UAT; do not assume it is an MSI or Windows service.
macOS Guard agent	Distribute the approved macOS archive or signed package through the bank's MDM. Keep the archive's related files together and run the supplied installer command or approved package workflow.	Confirm application signing/notarization, permissions, LaunchAgent/startup behavior, certificate trust and user prompts on the selected macOS release. Use MDM profiles for approved trust and proxy configuration. Validate agent heartbeat and a synthetic policy event before broad rollout.
Chrome, Edge and other Chromium browser extension	Deploy the release's `.crx` or enterprise extension package using managed browser policy and an internal extension distribution/update source.	Force-install or centrally manage the extension using the browser's enterprise policy. Confirm the release extension ID, allowed install source, update policy, agent hand-off, target matching and block/warn behavior. The unpacked/developer-mode package is for UAT diagnosis only, not production fleet rollout.
Firefox extension	Deploy the release's `.xpi` through the bank's managed Firefox extension policy after confirming the package is signed and compatible.	Confirm policy installation, update source, certificate trust, agent communication and event delivery on the exact managed Firefox build. Temporary add-on loading is for UAT only.
Safari	Treat Safari as an unqualified candidate until the selected release supplies and passes the required Safari packaging and policy path.	Verify extension availability, macOS system proxy behavior, certificate trust and Private Relay or other network features in UAT. Do not claim Safari support until this qualification is complete.

Prepare the Guard bundle so its non-secret configuration references the bank's internal Gateway name and approved policy mode. Keep shared secrets and signing material out of the extension package, employee instructions and general software repository; use the bank's protected endpoint configuration mechanism. Inspect the release's certificate-generation and key-custody model. Distribute only the approved trust material, protect any signing/private keys and define certificate renewal and revocation before rollout.

Install and qualify the agent before distributing the extension to a pilot cohort. The pilot verifies endpoint identity persistence, PAC/proxy selection, certificate trust, agent heartbeat, policy/rule synchronization, prompt interception, block/warn/redact behavior, Browser AI log delivery, update handling and uninstall/rollback. Roll out by managed endpoint groups, measure healthy-agent coverage and event delivery, then expand. In persistent and non-persistent VDI, verify whether identity and trust persist per user or per session and prevent identity collisions before adding the image to broad deployment.

If the agent, certificate or Gateway is unavailable, the effective behavior depends on the selected release and proxy policy. Confirm its fail-open/fail-closed behavior in UAT. Where policy requires that public AI never be reachable directly, enforce the denial at the bank endpoint/network boundary so an endpoint agent failure or PAC bypass cannot route around that rule.

Release promotion and rollback

Use UAT as the first deployment. Import the signed image, chart and Guard packages through internal repositories, validate database migration and private-service connectivity, then promote the same immutable digest and release-matched manifest to DR and DC with environment-specific secrets. Start with a pilot and expand after observing health, database pool pressure, Guard heartbeat/event delivery and internal egress-deny records.

Before an upgrade, preserve the approved configuration and a tested database/attachment recovery point. If health, integration or policy behavior regresses, halt expansion and route traffic to the previous approved image and compatible configuration. Use only a tested schema migration or restore path; never assume database downgrade is safe. Confirm agent/extension compatibility and certificate or key rotation through the bank's change process.

Feature and functional reference

This section presents the product behavior from an operator and application-owner perspective. The detailed component and request lifecycles remain in Sections 18 and 9.

Capability	How it works	Primary control or record
AI API gateway	Accepts supported inference requests, authenticates the caller, resolves the model/provider route, applies configured governance and guardrails, then returns or streams the provider result.	Virtual keys, provider/model configuration, routing and request logs.
Provider and model integration	Provider adapters translate supported request and response shapes. Capabilities differ by provider and operation; model selection must match the configured adapter.	Provider credentials, model catalog and route configuration.
Model catalog and model settings	Presents configured or discovered model metadata, model-specific settings, pricing and scoped limits where supported.	Model catalog, model settings, pricing overrides and provider configuration.
Routing and resilience	Supports configured routing rules, adaptive/load-balancer controls, circuit-breaker state and eligible provider fallback paths.	Routing rules, route weights, provider health and circuit-breaker configuration.
Governance	Associates callers with virtual keys, budgets, rate limits and organizational scope.	Virtual-key and governance records; configured 429 responses when limits are reached.
Organization and identity governance	Provides user lifecycle, role/resource permissions, section grants and organization/group scopes; optional identity provisioning integrations depend on configuration.	Users, roles, access profiles, teams/business units/customers and provisioning records.
Browser AI Guard	Uses a managed extension and endpoint proxy/PAC path to identify supported AI-site requests, evaluate active rules and apply the configured action.	Targets, rules, controls, agent/fleet state and policy events.
Prompt and file controls	Supports configured allow, redact, warn, block and site-block outcomes on supported request surfaces. Optional AI Guard Bot evaluation uses its configured model/service.	Guard rule/action, content extraction path and event record.

Capability	How it works	Primary control or record
Prompt and skills workspaces	Provides prompt and skills repositories, organization-scoped access and prompt lifecycle/deployment features where enabled.	Repository folders, version/history and deployment configuration.
Admin and policy management	Serves the management UI with the Gateway; management APIs validate sessions, RBAC and section grants, then persist supported changes and audit mutations.	Session, resource/operation grants, section grants and audit record.
Observability	Exposes configured health, logs, metrics and trace paths for internal operations. External connectors are optional and must be disabled or redirected for a no-egress deployment.	Health/readiness checks, internal collectors and logging configuration.
MCP Gateway	Optionally manages MCP clients, sessions, tool groups and tool calls. Each configured MCP server is a potential data destination.	MCP credentials, access configuration and MCP logs.
Extension and integration framework	Built-in plugin interfaces and optional connectors enable selected telemetry, governance, prompt, guardrail, caching and integration behaviors. Plugin availability and load method are release-specific.	Enabled plugin configuration, startup status and approved internal connector destinations.

User roles and permissions

The implementation seeds three dashboard roles. Custom roles and section grants can narrow access. The role names below describe the current application model; they do not replace the bank's identity lifecycle, approval or separation-of-duties policies.

Role or identity	Implemented access	Operational guidance
`admin`	Broad access to application data and every permission in the seeded catalog.	Reserve for named platform administrators. Review use through audit records.
`sub_admin`	Read/view access to dashboard, inference, observability, MCP logs, routing rules and metrics; broader log, virtual-key and prompt-repository actions; selected governance and MCP management. Provider secrets and system settings remain restricted by the seeded role.	Grant only the resource and section scopes needed for the operator's function. Browser AI write routes map to the Logs resource in the current implementation.
`user`	Own-data, read/view access to selected dashboard, logs, inference, prompt repository, observability and MCP views; no general administration rights.	Use allowed-section grants to limit what is visible.
Guard agent	Not a dashboard role. Uses the Guard fleet secret on the specific agent/event routes that support secret authentication.	Store and rotate the secret through protected endpoint configuration; do not place it in employee-visible material or a browser extension bundle.

Representative workflows

For an application request, a client presents its approved Gateway credential and a supported request shape. The Gateway authenticates and resolves the provider/model, applies configured routing and governance, calls the approved endpoint, returns the result and records only the configured logs and telemetry. The detailed sequence is in the inference lifecycle diagram.

For Browser AI Guard, a managed browser uses the configured extension and proxy path. The Guard checks the target and supported content surface, evaluates configured rules, applies the selected outcome and sends the event to the internal Gateway. Administrators review events through role-gated pages. Network egress policy remains the enforcement boundary if the endpoint agent or PAC path is unavailable.

Logical data design and lifecycle

The product separates configuration/workspace records from request/event records. This diagram shows logical associations only; it is not a physical foreign-key contract. Actual database names, schemas, hostnames, listener values and credentials are deployment-specific and remain in the bank's protected DBA records.

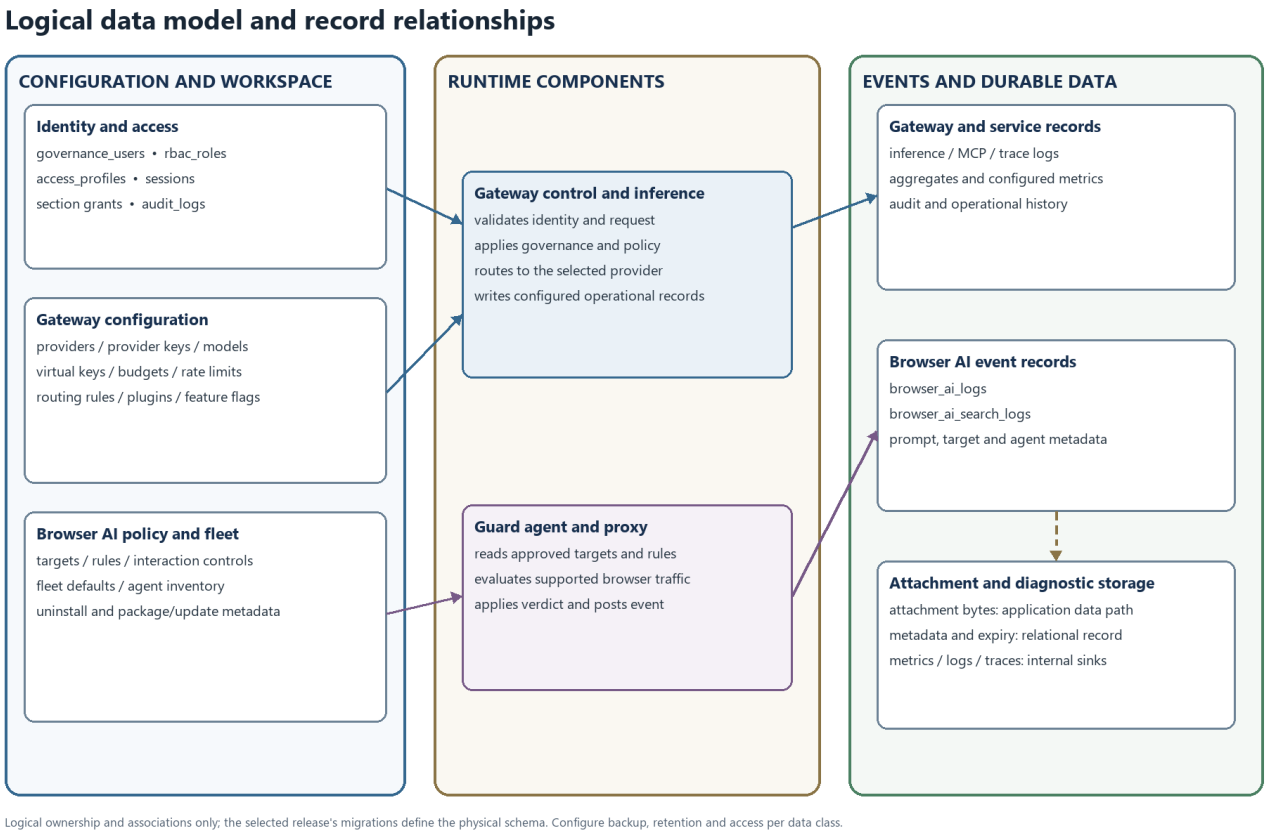


Figure 9. Logical data groups and associations. This view is conceptual and does not assert physical foreign-key constraints.

Data group	Representative records	Storage and handling
Identity and access	Users, roles, access profiles, sessions and audit records.	ConfigStore relational records. Protect administrative fields and preserve audit history according to bank policy.
Gateway configuration	Providers, provider keys, models, model settings, virtual keys, budgets, rate limits, routing rules and feature flags.	ConfigStore. Database encryption of selected stored secrets depends on the configured encryption key; use database/storage encryption as an additional control.
Guard policy and fleet	Targets, rules, interaction controls, fleet defaults, agent inventory and package/update metadata.	Browser AI configuration records. Restrict write access and protect the shared Guard secret.
Runtime records	Inference, MCP, Browser AI prompt/search events, trace/aggregate records and request metadata.	LogsStore and configured internal observability services. Prompt, URL, user and model metadata can be sensitive.
Attachments	Uploaded file bytes plus name, type, size, path and expiry metadata.	Bytes are stored under the configured application data directory; metadata is stored with Browser AI records. Back up, restore and delete them as a coordinated set.

Data group	Representative records	Storage and handling
Runtime diagnostics	Application logs, metrics, traces, denial records and health data.	Send only to bank-operated internal collectors for a no-egress design. Keep diagnostic access and retention separate from business audit records.

Section 8 lists the database model and key table families. Before upgrades, follow release migration notes, snapshot the database and attachment storage, and rehearse on restored UAT data. Do not assume a schema downgrade is safe.

Retention is configured by record class and store. Set and verify policies for inference/audit logs, Browser AI events, attachments, application logs, traces, backups and replicas. Cleanup controls vary by enabled feature; verify deletion across databases, attachments, backups and replicas because database deletion alone does not expire those copies.

API documentation and integrations

Sections 7 and 9 list routes. This section summarizes API authentication, request examples and integrations; exact routes and provider support depend on the selected release.

API surface	Typical caller	Authentication and authorization
`/v1/*` inference APIs	Internal application or approved SDK	Virtual key or configured provider key in the supported authorization header. Governance controls apply where configured.
Provider pass-through routes	Internal application using a supported compatibility surface	Gateway inference authentication, then the configured provider route and policy.
`/api/*` management APIs	Administrator or permitted operator	Dashboard session followed by resource/operation RBAC and applicable section grants. Mutations enter the audit path.
Browser AI Guard event routes	Managed Guard agent	Shared secret header `X-Gateway-Guard-Key`; production should require a configured secret. Event ingestion verifies the Guard security policy and applies request throttling.
Browser AI policy-read routes	Guard agent or administrator	Guard secret for the specific supported read routes, or an authorized dashboard session. Administrative writes require session/RBAC.
Health and version	Internal platform monitoring	Internal checks only; restrict diagnostic access at ingress and disable profiling in production.

Illustrative inference call

The following example uses synthetic content and environment-supplied values. It is an OpenAI-compatible request shape; the selected provider, model and operation determine exact support and response details.

```
curl --fail-with-body --request POST \
  "${BANK_INTERNAL_GATEWAY_URL}/v1/chat/completions" \
  --header "Authorization: Bearer ${VIRTUAL_KEY}" \
  --header "Content-Type: application/json" \
  --data-binary @synthetic-request.json
{
  "model": "<configured-model-id>",
  "messages": [
    {"role": "user", "content": "Synthetic UAT request: reply with the word approved."}
  ],
  "stream": false
}
```

An OpenAI-compatible response commonly contains a request identifier and a `choices` array with an assistant message. Other routes and provider adapters may use different fields or streaming event formats. Treat a timeout or mid-stream disconnect as an unknown completion state for operations that may create side effects.

Illustrative Guard event

Guard event calls use the fleet secret header and must be sent only to the bank's internal Gateway name. Never substitute a real prompt or file during API validation.

```
curl --fail-with-body --request POST \
  "${BANK_INTERNAL_GATEWAY_URL}/api/browser-ai/intercept" \
  --header "X-Gateway-Guard-Key: ${GATEWAY_GUARD_SECRET}" \
  --header "Content-Type: application/json" \
  --data-binary @synthetic-guard-event.json
{
  "platform": "synthetic-test-target",
  "prompt": "Synthetic UAT content only",
  "agent_id": "<test-agent-id>",
  "agent_hostname": "<test-host-label>",
  "metadata": {"evaluation_only": true, "domain": "<test-domain>"}
}
```

The response reports the policy outcome, such as `allowed`, `action`, `rule_triggered` and, where applicable, the forwarded/redacted prompt or evaluation status. Schema validation, policy mismatch, invalid secret, unavailable feature/store and throttling can return client, authorization, availability or server errors. The route-specific API contract in the selected release remains authoritative.

Errors quotas and integrations

Section 19 documents the error response shape, common HTTP status classes, retries, streaming behavior and recovery. Callers should preserve the request identifier, back off on throttling, avoid replaying non-idempotent requests and never retry a known invalid credential unchanged. Governance rate limits are configurable against supported owners such as virtual keys or providers; provider quotas also apply. There is no single fixed inference throughput limit in this manual. Guard route throttling and login protections are separate controls; confirm the effective release configuration instead of deriving a rate from the endpoint count.

Integration	Role	Data-residency decision
Model/provider endpoint	Inference, embeddings, image/audio/video or other supported operations.	Select a bank-hosted/private endpoint for no-egress operation. A public provider route sends request data outside the bank boundary and must be disabled or explicitly approved.
Identity and provisioning	Application sessions, OAuth2 flows and optional SCIM provisioning.	Use bank-operated identity endpoints and internal DNS/TLS; review claims, groups and provisioning permissions.

Integration	Role	Data-residency decision
Database and storage	ConfigStore, LogsStore, vector/cache services if enabled, and attachment storage.	Keep services inside the bank-controlled India boundary; use internal TLS, service identities and tested backup/restore.
Observability and alerts	Prometheus/OpenTelemetry-compatible paths, internal log collection, SMTP or configured alerts.	Disable external connectors or route to approved internal destinations; inspect labels and payload fields before export.
MCP servers	Optional tools and data sources.	Treat each server and tool as an outbound data path; allow only explicitly approved bank-operated destinations.

Security and compliance mapping

Control area	Product behavior	Deployment responsibility
Authentication	Dashboard sessions, supported OAuth2 flows, inference keys and Guard fleet-secret authentication protect different surfaces.	Integrate approved identity, control session policy, secure key distribution and review dormant accounts.
Authorization	Resource/operation RBAC and section grants are checked on management routes; inference governance can scope keys and limits.	Assign least privilege, test named roles and review administrative actions.
Encryption in transit	The application is intended to be placed behind an internal TLS ingress; database/provider TLS is configured per endpoint.	Terminate or pass through TLS under bank standards, validate certificates, restrict trusted proxy sources and require TLS for database traffic.
Encryption at rest	The application encrypts selected database secrets when `GATEWAY_ENCRYPTION_KEY` is configured. This does not establish application-level encryption of every log, prompt or attachment.	Enable database, volume, backup and key-store encryption; protect and recover encryption material independently.
Audit and logging	Mutating management APIs have an audit path; inference, MCP, Guard, metrics and traces are distinct record classes.	Restrict access, minimize payload capture, set retention and keep logs/traces internal. Redact secrets and sensitive content in support evidence.
No-egress operation	Private provider/model endpoints and internal collectors can be configured. The product cannot by itself stop a network path that the bank leaves open.	Enforce default-deny egress, internal mirrors, DNS controls and India-resident DR/backups at the network/platform layers.
Compliance evidence	This manual describes implementation controls and design responsibilities. It does not assert a certification or independent compliance assessment.	Map the deployed controls and evidence to the bank's applicable regulatory and assurance framework; confirm any ISO, privacy or other certification claims with formal evidence.

Deployment environments and infrastructure

Environment model and release pipeline

Use separate development, UAT, DR and DC configuration boundaries. UAT must have separate identities, secrets, data stores and model endpoints; use synthetic or approved masked data. DR holds the selected recovery topology and encrypted replication/backup flow. DC serves production traffic. Apply the same approved immutable release through UAT, DR and DC, with only protected environment configuration changing.

Environment	Purpose	Data and access boundary
Development	Engineering and component integration.	Synthetic test data and restricted developer identities; no production credentials.
UAT	Release, migration, policy, browser, performance and recovery qualification.	Separate stores and secrets; synthetic or approved masked data only.
DR	Recovery capacity and site failover/failback.	Bank-controlled replication and backup; access limited to recovery operators.
DC	Production service for approved applications and managed endpoints.	Production identities and data under bank controls, with monitoring and change restrictions.

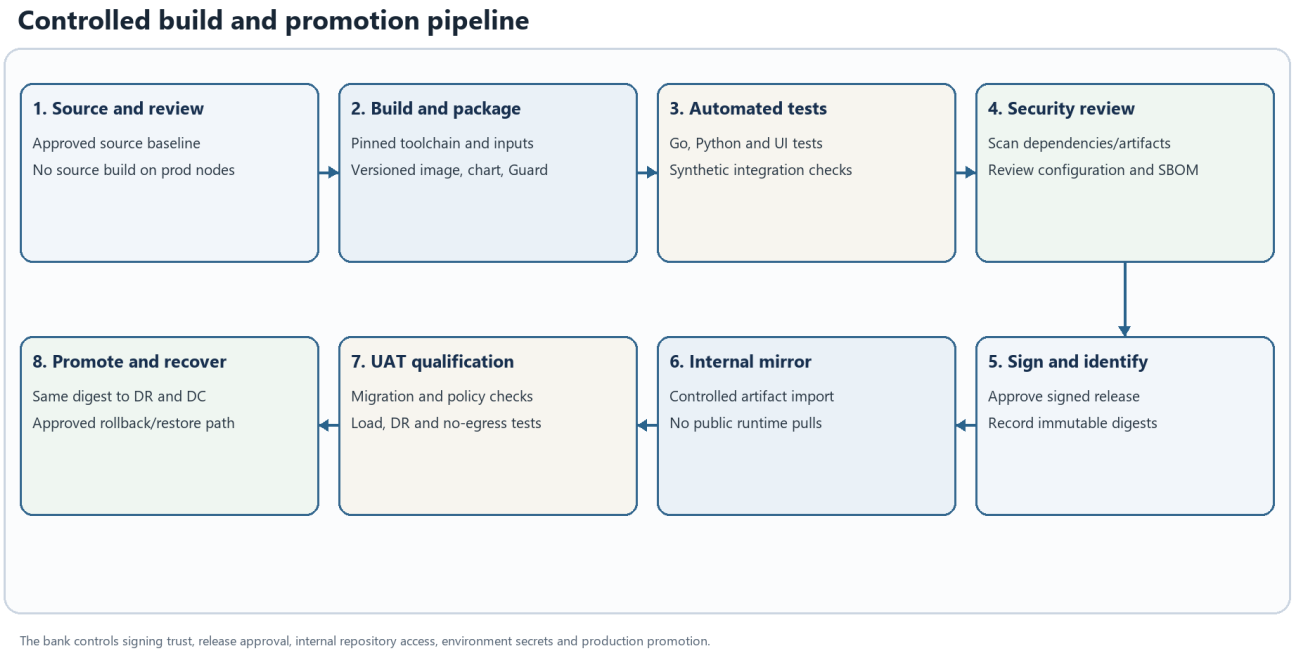


Figure 10. Controlled build and promotion pipeline. This is a logical workflow; enabled stages depend on deployment configuration.

The repository includes Docker and Helm packaging. Compose is suitable for isolated evaluation or a limited smoke deployment; it is not the reference 120,000-endpoint production topology. Production should use the bank's supported container platform, internal ingress/registry, release-matched manifests, persistent storage, resource controls, readiness checks, disruption protection and a bank-managed database. The deployment procedure, database requirements and endpoint packaging details are in Section 20.

Runtime prerequisites and compatibility boundaries

Area	Current implementation evidence	Qualification boundary
Gateway runtime	Container/Docker and Helm packaging; Go server with embedded administration UI.	Match the node OS/architecture, container runtime, Kubernetes/platform version and image digest to the selected release's support statement. No cloud provider is required for an on-prem runtime.
Database	PostgreSQL, MySQL and SQLite driver paths are present; PostgreSQL is the production planning baseline in this pack.	Validate exact engine/version/topology with the selected release. SQLite is not the HA database choice for the 120,000-endpoint design.

Area	Current implementation evidence	Qualification boundary
Endpoint agent	Windows and macOS packaging/build paths exist.	Verify exact OS editions/builds, signing, permissions, certificate deployment and endpoint-management behavior in UAT.
Browser extension	Separate Chromium `.crx` and Firefox `.xpi` packaging paths are documented.	Confirm browser version, managed install/update policy, extension identifier and feature behavior for each release. Do not infer certification from package presence.
Safari	No qualified Safari packaging and enterprise policy matrix is established here.	Treat as unsupported for the approved rollout until an exact release package and UAT qualification are documented.
VDI	Agent/proxy/PAC and certificate behavior can be exercised in virtual desktop environments.	No vendor/version certification matrix is established here. Qualify persistent and non-persistent profiles, identity persistence, certificate trust, user isolation and session cleanup.
Network and services	Internal DNS/NTP, TLS ingress, database, private model routes, storage, endpoint API access and optional telemetry paths.	Use logical service names and service-role allow-lists. Keep bank hostnames, numeric listener values and database identifiers in protected deployment records, not this client reference.

For a 120,000-endpoint program, use the companion on-premises architecture document for the DC/DR/UAT reference topology and planning sizing. This manual does not convert the endpoint count into a tested capacity guarantee. Confirm all component versions and deployment options against the release artifact and bank platform standard.

Performance and scalability

The repository does not provide a qualified benchmark for the 120,000-endpoint target. Endpoint inventory is not the same as simultaneous prompt traffic. Capacity depends on active endpoint share, prompt and attachment rate, request size, rule complexity, optional AI Guard Bot evaluation, model latency, agent heartbeat frequency, log retention, DB write/index cost and required availability margin.

Scale Gateway replicas horizontally only with the release-supported cluster/configuration behavior, an approved database topology, bounded connection pools and durable attachment storage. Calculate the total database connection budget across peak replicas, jobs, migration activity and operators. Size database IOPS, WAL/replication, indexes and storage from measured event volume and retention. Keep DC and DR failover capacity available for the chosen recovery mode; application replicas do not make the database highly available by themselves.

Treat semantic/response caches as data stores. Enable them only after reviewing cache keys, tenant separation, retention, encryption and locality. Disable cache or connector features that are not approved for the data class. Avoid unbounded retries, request-body logging and synchronized client retry bursts.

Qualification should measure sustained and burst request throughput, p50/p95/p99 latency, error/retry rates, provider concurrency, database pool and lock pressure, storage/backup growth, agent heartbeat and event-delivery coverage, and failover recovery. Use synthetic content and record workload shape, release digest, environment, topology and test duration with results. Publish measured numbers only for the tested configuration.

Monitoring backup and maintenance

The health endpoints check configured ConfigStore, LogsStore and VectorStore dependencies unless dependency pings are disabled. Keep process liveness separate from dependency readiness. Scrape metrics and receive logs/traces through bank-operated internal collectors. Use internal alerts for health failures, dependency latency, elevated 4xx/5xx/429 rates, retry growth, DB pool/replication pressure, storage exhaustion, agent heartbeat gaps, event ingestion gaps, certificate expiry and egress-deny anomalies. Avoid exporting prompt or credential labels.

Back up databases, attachments, application configuration, release artifacts and encryption/secret recovery material under coordinated schedules. Protect backups with bank-controlled encryption and access. Test restore and point-in-time recovery; verify restored attachment metadata/bytes and retention cleanup. Replication and backup targets must remain inside the approved India residency boundary.

For change and patch operations, promote an immutable image/chart and Guard package through UAT, compare release digests, review migrations, preserve a recoverable state and use gradual endpoint cohorts. Rotate database, provider, session-signing, encryption and Guard secrets through their supported procedures; do not embed them in source, extension packages, ordinary environment manifests or support files. If a key protects stored ciphertext, preserve an approved recovery path before rotation. Keep pprof and development routes unavailable in production.

Testing QA and known limitations

The repository contains Go tests for core/framework/plugins/HTTP handlers, Python tests for Guard agent and proxy behavior, UI tooling and integration/e2e targets. Test availability is not the same as a release qualification report; this manual makes no code-coverage or certification claim.

Test layer	Purpose
Unit and contract tests	Validate provider adapters, shared schemas, policy/routing logic, persistence and error mapping with synthetic requests.
Integration tests	Exercise Gateway-to-database, private model, identity/provisioning, internal observability and Guard event paths.
Endpoint/browser tests	Validate exact OS/browser builds, managed extension installation, PAC/proxy selection, certificate trust, allow/redact/warn/block outcomes, update, uninstall and VDI session behavior.
Security tests	Verify authentication, role/section boundaries, secret rejection/rotation, default-deny egress, disabled profiling, log minimization and prohibited direct AI access.
Performance and resilience tests	Measure representative concurrency, burst, payload and retention loads; exercise dependency loss, streaming interruption, DB failover, DR promotion/failback and restore.

Known boundaries: provider operations and stream behavior differ; a complete release/browser/VDI certification matrix is not established here; no benchmark guarantee is claimed; Guard inspection depends on supported request extraction and correct managed proxy/extension deployment; an agent or PAC failure may permit direct traffic unless the network blocks it; payload capture, optional AI evaluation, MCP and connectors can create additional data paths; and at-rest encryption of all event and attachment data is not implied by database-secret encryption. Apply explicit network, storage, retention and release-specific tests for the bank's data class.

Troubleshooting and support

Section 19.5 contains the detailed operational recovery matrix. The following answers summarize the first diagnostic path.

Question or symptom	First investigation
The health route is unavailable or returns an error.	Compare process liveness with dependency readiness; inspect ConfigStore/LogsStore/VectorStore health, TLS, credentials and connection pools.
A valid inference request returns an authorization or quota error.	Check the virtual key, model/provider grants, budget/rate settings, provider credential and request identifier. Do not expose credentials in diagnostic output.

Question or symptom	First investigation
Guard heartbeat succeeds but prompts are not blocked.	Check target matching, rule scope/order, agent health, extension hand-off, PAC route, certificate status and direct network egress policy. Test with synthetic content.
Guard events are missing.	Check secret configuration, internal event route, LogsStore health, endpoint connectivity, retention filters and time synchronization.
An attachment cannot be opened after restore.	Compare database metadata and expiry with the restored application-data volume, ownership, permissions and storage path.
Streaming stops after content begins.	Treat partial output as incomplete; preserve the request ID, inspect provider and ingress timeout, and do not replay side-effecting operations without reconciliation.

For escalation, use the support channel and service hours defined in the bank's product agreement. Include the product release and artifact digest, site/environment, UTC timestamp, request or agent identifier, route, status, sanitized error and relevant health indicators. Use synthetic reproductions where possible. Remove prompts, file contents, access tokens, secret headers and private host details unless the bank has approved a protected evidence-transfer path.

Appendix configuration diagrams and roadmap

Environment configuration inventory example

The following is an inventory template, not a ready-to-run `.env` file or Helm values file. Resolve each value from the bank's protected configuration and secret-management process. Keep actual internal addresses, listener values, database identifiers and credentials outside this client document.

```
APP_HOST=<bank-approved bind setting>
APP_DIR=<durable application-data mount>
DB_TYPE=<release-supported database engine>
DB_HOST=<protected internal database name>
DB_PORT=<bank-managed listener value>
DB_NAME=<environment-specific database identifier>
DB_USER=<secret-manager reference>
DB_PASSWORD=<secret-manager reference>
DB_SSL_MODE=<release-supported verified TLS mode>
GATEWAY_ENCRYPTION_KEY=<bank-controlled key reference>
GATEWAY_GUARD_SECRET=<secret-manager reference>
GATEWAY_GUARD_REQUIRE_SECRET=1
GATEWAY_METRICS_TOKEN=<secret-manager reference>
```

Use only configuration names supported by the selected release's environment template, chart and loader. Separate UAT, DR and DC secret scopes; keep secret values out of source control, command history, browser extension packages, tickets and general support bundles.

Diagram index

The manual includes the consolidated control/data/persistence/observability architecture; inference, Browser AI Guard, administration, MCP, settings/observability and error-recovery workflows; the logical data model; and the controlled release path. The On-Premises Deployment Architecture contains the separate DC/DR/UAT topology, India residency/data-flow boundary, denied-egress paths and site rollout. Use each figure with its nearby assumptions; all diagrams are logical and must be reconciled with the selected release and bank network design.

Roadmap and future enhancements

This manual documents current implementation paths and makes no future delivery or date commitments. Product roadmap priorities and planned enhancements are governed separately by the product owner and should be reviewed through the applicable product planning process.

End user quick manual

This guide is for employees and application users. The bank administrator supplies the internal workspace address, approved model choices, managed-browser setup and support channel. Available pages and actions depend on the user's role and section grants.

Employee using a managed browser

1. Sign in to the bank-managed workstation or VDI session and use a browser that the endpoint team has qualified and configured for YP-Raksha Guard.
2. Use only AI websites and destinations approved by the bank. A visible Guard extension does not make an unapproved public site safe or approved.
3. Before submitting, remove unnecessary customer, payment, credential, health, identity and confidential data. Follow the bank's data classification and AI-use policy.
4. Read the in-browser outcome. **Allow** continues the request; **redact** submits the configured modified content; **warn** presents the configured policy notice; **block** stops the request; **site-block** prevents access to the configured site. Exact wording and warn confirmation behavior depend on the release and policy.
5. If a request is blocked or redacted unexpectedly, stop and contact the bank support channel with the time, site, browser/VDI type and a short description. Do not send the original sensitive prompt, attachment, session cookie or a screenshot containing protected data unless the security team explicitly requests an approved evidence channel.

Do not disable the agent/extension, change proxy settings, install an unapproved certificate or retry through a personal device to bypass a policy. If the Guard is unavailable, use the bank's approved fallback process; direct access behavior is controlled by endpoint and network policy.

Employee using the YP Raksha workspace

Open the internal address provided by the bank. Sign in using the assigned account and authentication flow. The navigation menu shows only the pages granted to that account. Use the approved model/provider options exposed to your role, and do not copy API keys into source code, browser storage, tickets or shared documents.

If the bank enables the Prompt Repository, use its approved folders and sharing controls. If access is denied or a page is absent, request the minimum required permission from the designated administrator; do not use another employee's account. When reporting an issue, provide the time, page or API operation, sanitized error/status and request identifier if available.

Application developer using the Gateway API

Obtain an application-scoped virtual key and approved model/route from the platform owner. Configure the supported Gateway base address and key through the application secret store. Use the API examples in Section 24 with synthetic content in UAT first, honor returned status and rate-limit guidance, set finite timeouts, and redact keys and prompt bodies from application logs. A request that times out after submission can have an unknown completion state; do not automatically repeat a side-effecting operation without reconciliation.

Quick troubleshooting

User-visible issue	Safe user action	Information to share with support
Sign-in denied or session expired	Sign in again once; if still denied, request account support	Time, sanitized message and whether the issue affects other internal services
Model or route unavailable	Stop repeated attempts and use only an approved alternate model	Request ID, selected model label, status and time; never send the key
Rate or budget limit reached	Wait or request an approved limit review	Application/team name, request ID and time
Browser warning/block	Follow the displayed policy and do not bypass Guard	Site, browser/VDI type, time and sanitized rule/event reference
Slow or interrupted stream	Close the incomplete operation and follow the application retry policy	Request ID, time, route and sanitized client/network error

Administrator operating manual

This operating guide complements the deployment procedure in Section 20, the management API reference in Sections 7 and 24, and the bank-specific site design. Exact UI labels and available actions can change by release; use the matched release UI and API schema.

Establish an environment

Deploy first into isolated UAT using the release-matched configuration, signed Gateway and Guard artifacts, a separate database, separate secrets and a private test model. Replace development defaults, verify internal DNS/TLS and identity, confirm that the Gateway is ready, and check the selected health/readiness route. Never place actual bank addresses, credentials or key values in this manual or a general support bundle.

For production, use the bank platform's supported container/Kubernetes service, internal ingress, durable application/attachment storage, HA relational database, internal artifact mirror, secret manager and internal observability destinations. Keep UAT, DC and DR identities and secrets separate. Section 20 gives the environment variables and database preparation sequence; the architecture companion gives the site resource design.

Secure administrator access and roles

Create named administrator identities through the approved identity lifecycle. Assign the broad `admin` role only to operators who require full control; use `sub_admin`, custom roles, resource-operation grants and section grants to scope routine tasks. Review session handling, trusted ingress/proxy settings, audit records and emergency access with the bank identity/security owners. The built-in application role model is not a substitute for the bank IdP, MFA or separation-of-duties control.

Provision and disable accounts through the approved local, directory or SCIM path supported by the selected release. Where the workflow is enabled, review pending/approved/rejected/disabled states, map users to the approved team/business unit/customer scope, apply only the required access profile and sidebar sections, and verify with a representative named test account. Configure password-reset mail and notifications only when a bank-controlled identity and internal mail path are available. Do not rely on sample bootstrap credentials or public email defaults.

Configure provider model and application access

1. Add only the approved private provider endpoint and the minimum provider credentials needed. Protect credentials using the selected release's encryption configuration and the bank secret process.
2. Register the required models and operations, then validate a synthetic request for each required API shape, streaming mode and provider-specific feature.
3. Create application- or team-scoped virtual keys. Set approved budgets, rate limits and model scope; associate only the intended users or teams; store the returned secret in the application secret manager and distribute it through the approved channel.
4. Configure governance, routing, fallback and circuit-breaker behavior only for approved destinations. Confirm that fallback never routes sensitive prompts to a public or out-of-residency provider.
5. Review inference logs and audit separately. Minimize prompt/response capture, restrict access and set retention consistent with bank policy.

Configure Browser AI Guard

1. Review the exact target domains and host roles that must be monitored, blocked or excluded. Include the UI, chat, upload and API hostnames required by the target service.
2. Define each Guard rule's match pattern, target scope, action, priority and exception owner. Keep a documented difference between browser Guard rules and Gateway API guardrails; they protect separate request paths.

3. Set fleet controls, agent authentication, proxy/PAC configuration, certificate trust and approved update source using protected configuration. Keep shared Guard secrets out of extension packages and employee-visible configuration.
4. Package and distribute the signed Windows/macOS agent and browser extension through the bank endpoint-management system. Validate identity persistence, proxy reachability, certificate trust, extension policy, heartbeat and update behavior on representative devices and VDI images.
5. Exercise allow, redact, warn, block and site-block on synthetic prompts and permitted test attachments. Check target matching, unsupported content paths, direct-egress denial, event ingestion and the administrator's role-scoped review view.
6. Define false-positive handling, emergency rule changes, agent rollback and secret rotation. Confirm the endpoint/network outcome when the agent, proxy, Gateway or event API is unavailable.

Section 20.6 describes endpoint package/extension installation and the Guard API paths; Section 18 contains the policy and event diagrams. A policy event record can include sensitive prompt, URL, file and user context depending on the configured capture path.

Configure observability retention and service ownership

Expose health checks and internal-only metrics to the approved monitoring segment. Route application logs, audit records, traces, alerts and database diagnostics to bank-operated destinations. Review sensitive labels and request attributes before enabling an exporter; keep public SaaS connectors, webhooks and external log sinks disabled for a strict no-egress design. Configure retention separately for inference, Browser AI, audit, application, trace, attachment, replica and backup data.

Assign an owner for Gateway service health, provider/model availability, database and attachment storage, Guard fleet health, certificates/secrets, internal collectors, endpoint rollout and network egress policy. The Observability flow in Section 18.6 shows the settings-to-collector path; the error matrix in Section 19 supports incident triage.

Change backup restore and recovery operations

Promote the same release digest from UAT to DR/DC, changing only protected environment configuration. Rehearse database migrations, rule changes, certificate and secret rotation, Guard updates and rollback in UAT. Take coordinated recovery points for the database and attachment storage; protect encryption keys and Guard secrets in the bank's recovery process. Test restore and site failover with the DBA, platform, network, model and endpoint owners and record measured recovery results in the bank's controlled operations records.

For a service incident, preserve the request/event identifiers and timestamps, check Gateway and dependency health, database pools/replication, model reachability, internal egress-deny events and Guard heartbeat/event delivery. Use synthetic examples when escalating and share raw prompt or attachment evidence only through an approved restricted channel.

Manage workspaces and optional integrations

For Prompt Repository and Skills Repository, assign folder/repository visibility to the intended user groups, review version/history and deployment behavior where enabled, and test access with both an authorized and a denied account. Treat saved prompts, skills and deployment content as bank data with an owner, retention expectation and approved sharing scope.

For MCP, register only approved bank-operated servers, review transport and authentication, restrict tool groups and user/session grants, and inspect the data each tool can read or write. For plugins, connectors, mail and alert channels, enable only required modules and destinations. A connector can export logs or event content; for no-egress operation, route it only to internal bank services or keep it disabled. Verify the resulting destination with network controls and internal records.